

# TPLA: TENSOR PARALLEL LATENT ATTENTION FOR EFFICIENT DISAGGREGATED PREFILL & DECODE INFERENCE

Xiaojuan Tang<sup>1,3\*</sup>, Fanxu Meng<sup>1,3\*</sup>, Pingzhi Tang<sup>1</sup>, Yuxuan Wang<sup>1</sup>, Di Yin<sup>3</sup>, Xing Sun<sup>3</sup>, Muhan Zhang<sup>1,2†</sup>

<sup>1</sup>Institute for Artificial Intelligence, Peking University

<sup>2</sup>State Key Laboratory of General Artificial Intelligence, BIGAI

<sup>3</sup>Tencent Youtu Lab, Shanghai, China

<https://github.com/fxmeng/TransMLA>

## ABSTRACT

Multi-Head Latent Attention (MLA), introduced in DeepSeek-V2, compresses key-value states into a low-rank latent vector  $\mathbf{c}^{KV}$ , caching only this vector to reduce memory. In tensor parallelism (TP), however, attention heads are computed across multiple devices, and each device must load the full  $\mathbf{c}^{KV}$ , eroding the advantage of MLA over Grouped Query Attention (GQA). We propose Tensor-Parallel Latent Attention (TPLA): a scheme that partitions both the latent representation and each head’s input dimension across devices, performs attention independently per shard, and then combines results with an all-reduce. TPLA preserves the benefits of a compressed KV cache while unlocking TP efficiency. Unlike Grouped Latent Attention (GLA), every head in TPLA still leverages the full latent representation, maintaining stronger representational capacity. TPLA is drop-in compatible with models pre-trained using MLA: it supports MLA-style prefilling and enables efficient tensor-parallel decoding without retraining. Applying simple orthogonal transforms—e.g., the Hadamard transform or PCA—before TP slicing further mitigates cross-shard interference, yielding minimal accuracy degradation. By reducing the per-device KV cache for DeepSeek-V3 and Kimi-K2, we achieve  $1.79\times$  and  $1.93\times$  speedups, respectively, at a 32K-token context length while maintaining performance on commonsense and LongBench benchmarks. TPLA can be implemented with FlashAttention-3, enabling practical end-to-end acceleration.

## 1 Introduction

Currently, large language models (LLMs) [1, 2, 3, 4, 5] are typically memory-bound (limited by memory bandwidth) rather than compute-bound (limited by floating-point operations per second, FLOPs) during inference. To address this, KV cache compression [6, 7, 8, 9] and tensor parallelism [10, 11, 12, 13, 14] have emerged as two critical techniques for enabling efficient auto-regressive decoding in LLMs. KV cache compression methods prune/merge/share/quantize intermediate key-value pairs to reduce memory overhead. Tensor parallelism addresses memory and compute limitations by splitting large tensors—such as weight matrices—across multiple devices, enabling intra-layer parallel computation for models that cannot fit on a single GPU. GQA [15] inherently supports both KV cache compression and tensor parallelism by grouping query heads so that all heads within a group share a common set of key and value representations, which facilitates efficient distribution across multiple devices. Both theoretical analyses and empirical results demonstrate that the representational capacity of GQA is inferior to that of MLA [16, 17]. MLA introduces a pre-trained KV cache compression strategy that achieves an excellent trade-off between computational efficiency and model performance. However, when multiple attention heads are computed in parallel across multiple devices using tensor parallelism, MLA encounters a critical limitation: each device must load the full latent vector  $\mathbf{c}_{KV}$ , undermining the memory savings that MLA offers over GQA. For example, in LLaMA-3-70B [18], the dimension of the KV cache per token is  $2 \times 8 \times 128 = 2048$ , and under tensor parallelism with  $TP = 4$ , each device holds a partitioned KV cache of size 512. In contrast, Deepseek-V3 [19] has a fixed KV cache dimension of  $64 + 512 = 576$ , which must be fully

\*Equal contribution.

†Corresponding author: [muhan@pku.edu.cn](mailto:muhan@pku.edu.cn)

# TPLA: 用于高效分解预填充和解码推理的张量并行潜在注意力

唐晓娟<sup>1,3\*</sup>、孟繁旭<sup>1,3\*</sup>、唐平志<sup>1</sup>、王雨轩<sup>1</sup>、尹笛<sup>3</sup>、孙兴<sup>3</sup>、张木涵<sup>1,2</sup>

†

1北京大学人工智能研究院 2通用人工智能国家重点实验室, BIGAI 3腾讯优图实验室, 中国上海 <https://github.com/fxmeng/TransMLA>

抽象的

DeepSeek-V2 中引入的多头潜在注意力 (MLA) 将键值状态压缩为低秩潜在向量  $c^{KV}$ , 仅缓存该向量以减少内存。然而, 在张量并行 (TP) 中, 注意力头是跨多个设备计算的, 并且每个设备必须加载完整的  $c^{KV}$ , 从而削弱了 MLA 相对于分组查询注意力 (GQA) 的优势。我们提出张量并行潜在注意力 (TPLA): 一种跨设备划分潜在表示和每个头的输入维度的方案, 对每个分片独立执行注意力, 然后将结果与全归约相结合。TPLA 保留了压缩 KV 缓存的优势, 同时释放了 TP 效率。与分组潜在注意力 (GLA) 不同, TPLA 中的每个头仍然利用完整的潜在表示, 保持更强的表示能力。TPLA 与使用 MLA 预训练的模型直接兼容: 它支持 MLA 式预填充, 并无需重新训练即可实现高效的张量并行解码。在 TP 切片之前应用简单的正交变换 (例如 Hadamard 变换或 PCA) 可进一步减轻跨分片干扰, 从而将精度下降降至最低。通过减少 DeepSeek-V3 和 Kimi-K2 的每设备 KV 缓存, 我们在 32K 令牌上下文长度下分别实现了  $1.79\times$  和  $1.93\times$  加速, 同时保持了常识和 LongBench 基准的性能。TPLA 可以通过 FlashAttention-3 来实现, 从而实现实用的端到端加速。

## 1 简介

目前, 大型语言模型 (LLM) [1,2,3,4,5] 在推理过程中通常受内存限制 (受内存带宽限制), 而不是受计算限制 (受每秒浮点运算数 FLOP 限制)。为了解决这个问题, KV 缓存压缩 [6,7,8,9] 和张量并行性 [10,11,12,13,14] 已成为在 LLM 中实现高效自回归解码的两种关键技术。KV 缓存压缩方法修剪/合并/共享/量化中间键值对以减少内存开销。张量并行性通过在多个设备上分割大张量 (例如权重矩阵) 来解决内存和计算限制, 从而为无法适应单个 GPU 的模型提供层内并行计算。GQA [15] 通过对查询头进行分组来本质上支持 KV 缓存压缩和张量并行性, 以便组内的所有头共享一组公共的键和值表示, 这有助于跨多个设备的高效分发。理论分析和实证结果都表明, GQA 的表征能力不如 MLA [16, 17]。MLA 引入了预训练的 KV 缓存压缩策略, 在计算效率和模型性能之间实现了良好的权衡。然而, 当使用张量并行性跨多个设备并行计算多个注意力头时, MLA 遇到了一个关键限制: 每个设备必须加载完整的潜在向量  $c_{KV}$ , 从而破坏了 MLA 相对于 GQA 提供的内存节省。例如, 在 LLaMA-3-70B [18] 中, 每个 token 的 KV 缓存维度为  $2 \times 8 \times 128 = 2048$ , 在 TP = 4 的张量并行下, 每个设备保存大小为 512 的分区 KV 缓存。相比之下, Deepseek-V3 [19] 的固定 KV 缓存维度为  $64 + 512 = 576$ , 必须完全

\*Equal contribution.

†Corresponding author: [muhan@pku.edu.cn](mailto:muhan@pku.edu.cn)

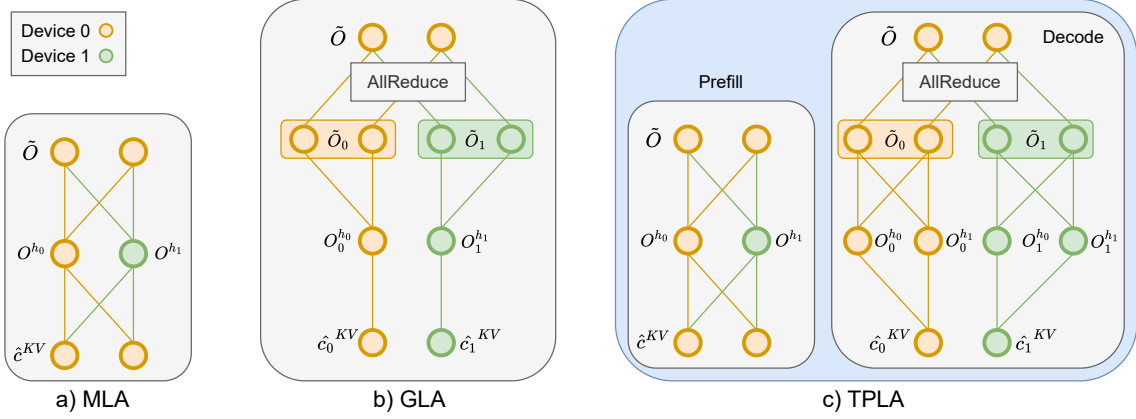


Figure 1: Comparison of MLA, GLA, and TPLA. In MLA, each device must load the entire KV cache. In GLA, each attention head only accesses the portion of the KV cache stored on its own device. In TPLA, the prefilling phase follows MLA for efficiency and accuracy, while during the decoding phase, attention heads are distributed across devices, each relying on the KV cache stored locally on its assigned device.

replicated on each device regardless of the parallelism degree. This results in a higher per-device KV cache memory footprint compared to GQA-based models under the same tensor parallel configuration.

GLA [20] was proposed to address the tensor parallelism limitations of MLA by dividing the attention heads and latent representations into  $g$  groups (typically  $g = 2$ ), such that each group of heads only loads its corresponding latent representation. However, this paper identifies two key limitations of GLA: (1) the reduction in KV cache size for single device comes at the cost of decreased representational capacity for each attention head; and (2) GLA requires training from scratch, which demands significant computational resources to validate its effectiveness.

To address these challenges, we propose Tensor Parallel Latent Attention (TPLA), a method that distributes the latent representations across multiple devices. Each attention head is split across devices, followed by an all-reduce operation on the output  $o$ . TPLA offers the following advantages: 1) Each attention head utilizes the full latent representation, preserving strong representational capacity; 2) Each device only loads a partition of the KV cache, improving inference speed under tensor parallelism; 3) TPLA can directly load pre-trained DeepSeek checkpoints, which incurs only a minor performance drop that is easily recovered; 4) We use reparameterized MLA for prefill and TPLA for decoding, reducing prefill latency while mitigating conversion-induced degradation. 4) TPLA can be viewed as a special case of GLA with more attention heads, making it compatible with FlashAttention-3.

## 2 Related Works

**Reducing KV-Cache Memory** Generative inference with large language models (LLMs) is often constrained by the memory footprint of the key-value (KV) cache, especially for long contexts. Several families of techniques have been explored to mitigate this burden: **token pruning/evicting** [21, 22, 23, 24, 25, 26] removes KV entries for low-importance tokens based on saliency or attention estimates; **token merging** [27, 28, 29] aggregates nearby or similar tokens into a single surrogate KV representation to eliminate redundancy while retaining context; **cross-layer KV sharing/fusion** [30, 31, 32, 33, 34] reuses one KV cache across adjacent layers to avoid per-layer storage; **low-rank KV compression** [35, 36, 37, 6] factorizes KV matrices into low-rank components (learned or SVD-based) to reduce dimensionality and memory; and **KV-cache quantization** [38, 39, 40, 41] stores K/V tensors at reduced numeric precision (e.g., int8 or int4), cutting memory and bandwidth with modest accuracy cost. Although effective, these approaches inevitably discard or alter information in the KV cache and can degrade model performance. In contrast, **TPLA** leaves the KV contents intact: it reduces the amount of cache each device must hold so the model retains full information while alleviating memory pressure. As a result, TPLA tends to preserve accuracy better than compression-based methods.

**Parallelism Strategies for Deployment** Current LLMs scale to billions of parameters; to cope with the resulting memory and compute demands, engineers adopt distributed deployment to reduce wall-clock latency and time costs. **Data parallelism** [42, 43] partitions input data across the sample or batch dimension while replicating model parameters across devices. However, for very large models full replication becomes impractical; moreover, variable sequence

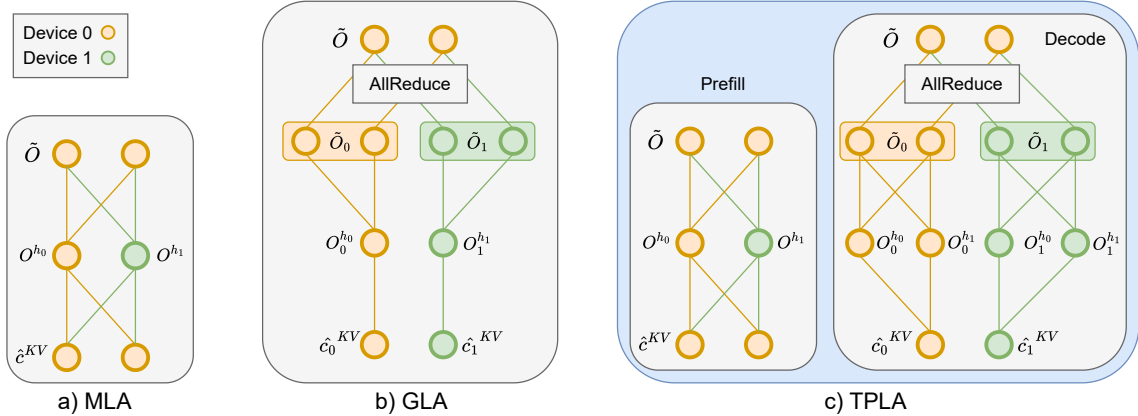


图 1: MLA、GLA 和 TPLA 的比较。在 MLA 中，每个设备必须加载整个 KV 缓存。在 GLA 中，每个注意力头仅访问存储在其自己设备上的 KV 缓存部分。在 TPLA 中，为了效率和准确性，预填充阶段遵循 MLA，而在解码阶段，注意力头分布在设备之间，每个设备都依赖于本地存储在其指定设备上的 KV 缓存。

无论并行度如何，都会在每个设备上复制。与相同张量并行配置下基于 GQA 的模型相比，这会导致每设备 KV 缓存占用空间更高。

GLA [20] 被提出通过将注意力头和潜在表示划分为  $g$  组（通常为  $g = 2$ ）来解决 MLA 的张量并行性限制，使得每组头仅加载其相应的潜在表示。然而，本文指出了 GLA 的两个关键限制：（1）单个设备 KV 缓存大小的减少是以每个注意力头的表示能力下降为代价的；（2）GLA 需要从头开始训练，这需要大量的计算资源来验证其有效性。

为了应对这些挑战，我们提出了张量并行潜在注意力（TPLA），这是一种跨多个设备分布潜在表示的方法。每个注意力头被分割到不同的设备上，然后对输出  $o$  进行全归约操作。TPLA 具有以下优点：1) 每个注意力头都利用完整的潜在表示，保持强大的表示能力；2) 每个设备只加载 KV 缓存的一个分区，提高张量并行下的推理速度；3) TPLA 可以直接加载预先训练的 DeepSeek 检查点，这只会导致轻微的性能下降，并且很容易恢复；4) 我们使用重新参数化的 MLA 进行预填充，使用 TPLA 进行解码，减少预填充延迟，同时减轻转换引起的性能下降。4) TPLA 可以看作是 GLA 的特例，具有更多的注意力头，使其与 FlashAttention-3 兼容。

## 2 相关作品

减少 KV 缓存内存大型语言模型 (LLM) 的生成推理通常受到键值 (KV) 缓存的内存占用的限制，尤其是对于长上下文。人们已经探索了几种技术来减轻这种负担：令牌修剪/驱逐 [21, 22, 23, 24, 25, 26] 根据显着性或注意力估计删除低重要性令牌的 KV 条目；令牌合并 [27, 28, 29] 将附近或相似的令牌聚合成单个代理 KV 表示，以消除冗余，同时保留上下文；跨层 KV 共享/融合 [30, 31, 32, 33, 34] 跨相邻层重用 KV 缓存以避免每层存储；低秩 KV 压缩 [35, 36, 37, 6] 将 KV 矩阵分解为低秩分量（学习的或基于 SVD 的）以减少维度和内存；KV 缓存量化 [38, 39, 40, 41] 以降低的数值精度（例如 int8 或 int4）存储 K/V 张量，以适度的精度成本削减内存和带宽。尽管有效，但这些方法不可避免地会丢弃或更改 KV 缓存中的信息，并可能降低模型性能。相比之下，TPLA 完整保留了 KV 内容：它减少了每个设备必须保存的缓存量，因此模型可以保留完整信息，同时减轻内存压力。因此，TPLA 往往比基于压缩的方法更好地保持精度。

部署的并行策略 当前的大语言模型已发展到数十亿个参数；为了应对由此产生的内存和计算需求，工程师采用分布式部署来减少挂钟延迟和时间成本。数据并行性 [42, 43] 在样本或批次维度上对输入数据进行分区，同时跨设备复制模型参数。然而，对于非常大的模型，完全复制变得不切实际；此外，可变序列

lengths introduce load imbalance (“bubbles”) that waste compute resources. **Pipeline parallelism** [44, 45] partitions the model into contiguous blocks of layers, each placed on a different device. Intermediate activations and gradients are communicated between stages to complete the forward and backward passes, reducing cross-node traffic. This staging overlaps computation across devices to increase throughput, but pipeline bubbles can still leave some devices idle. **Tensor parallelism** [46, 47, 48] splits linear layers along their row or column dimensions, sharding tensors across devices and performing distributed matrix–matrix multiplication with collective communication. It achieves optimal performance on systems where GPUs are fully interconnected via NVLink. TPLA leverages the strengths of TP while addressing MLA’s inability to reduce the KV cache under TP. Long sequences inflate the memory footprint of intermediate activations; **sequence parallelism** [49] mitigates this by replicating the model across devices and splitting inputs along the sequence dimension so that each device processes only a subsequence. **Prefill/Decode Separation** [50, 51, 52] refines sequence parallelism for LLM inference: the prefilling phase is compute-intensive and thus compute-bound, whereas the decoding phase has low per-token compute but frequent memory accesses and is memory-bandwidth-bound. To match these characteristics, different machine counts and architectures are used across the two phases to improve latency and throughput. In TPLA, we further employ different model structures across phases—MLA during prefill to preserve accuracy while reducing computation (improving latency), and TPLA during decoding to reduce memory traffic and increase throughput.

### 3 Preliminary

#### 3.1 Multi-Head Latent Attention

MLA is designed to reduce memory bandwidth overhead by compressing the Key-Value (KV) cache. Specifically, the multi-head keys and values are compressed into a single low-rank latent representation of dimension  $4d_h$ , denoted as  $\mathbf{c}^{\text{KV}}$ . Instead of reconstructing full-size keys and values from this latent representation, MLA adopts a more efficient decoding strategy. By isolating the Rotary Position Embedding (RoPE) operation, the up-projection matrix can be absorbed into the query activations, yielding  $Q$ . Similarly, the value projection is absorbed into the output projection matrix, resulting in  $W^{VO}$  (See Section 3.3). This allows for direct attention computation between  $Q$  and the normalized latent cache  $\hat{\mathbf{c}}^{\text{KV}}$ , followed by a projection through  $W^{VO}$  to produce the final output  $\tilde{O}$ . For simplicity in this initial description, we omit the RoPE components. The core computation is as follows:

$$\begin{aligned} \mathbf{c}^{\text{KV}} &\in \mathbb{R}^{B \times L \times 4d_h}, \quad \hat{\mathbf{c}}^{\text{KV}} = \text{RMSNorm}(\mathbf{c}^{\text{KV}}) \in \mathbb{R}^{B \times L \times 4d_h}, \\ Q &\in \mathbb{R}^{B \times 1 \times h_q \times 4d_h}, \quad W^{VO} \in \mathbb{R}^{(h_q \cdot 4d_h) \times D}, \\ O &= \text{softmax}\left(\frac{Q(\hat{\mathbf{c}}^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 4d_h}, \\ \tilde{O} &= O W^{VO} \in \mathbb{R}^{B \times 1 \times D}. \end{aligned} \tag{1}$$

$$\tag{2}$$

#### 3.2 Grouped Latent Attention

During tensor-parallel decoding, MLA replicates its single latent head on every device, resulting in high KV-cache memory load across all devices. GLA avoids this replication by partitioning the latent KV cache itself. Consider a two-way tensor-parallel configuration. The latent KV cache is divided into two shards,  $\mathbf{c}_0^{\text{KV}}$  and  $\mathbf{c}_1^{\text{KV}}$ , each assigned to one GPU. Simultaneously, attention heads  $h_q$  are grouped such that the absorbed query projection matrix  $Q$  and output projection matrix  $W^{VO}$  are partitioned along both the head dimension ( $h_q$ ) and the feature dimension ( $4d_h$ ), yielding four groups. Thus, GPU 0 operates on  $(\mathbf{c}_0^{\text{KV}}, Q_{0,0}, W_{0,0}^{VO})$ , while GPU 1 operates on  $(\mathbf{c}_1^{\text{KV}}, Q_{1,1}, W_{1,1}^{VO})$ . Each GPU independently computes its local attention output, denoted  $\tilde{O}_0$  and  $\tilde{O}_1$ , respectively. The final output is obtained via an AllReduce operation that sums the local outputs across devices.

$$\begin{aligned} \mathbf{c}_0^{\text{KV}}, \mathbf{c}_1^{\text{KV}} &\in \mathbb{R}^{B \times L \times 2d_h}, \quad \begin{cases} \hat{\mathbf{c}}_0^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_0^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \\ \hat{\mathbf{c}}_1^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_1^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \end{cases} \\ Q_{i,j \in \{0,1\}} &\in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \quad \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix} = Q, \\ W_{i,j \in \{0,1\}}^{VO} &\in \mathbb{R}^{\left(\frac{h_q}{2} \cdot 2d_h\right) \times D}, \quad W^{VO} = \begin{pmatrix} W_{0,0}^{VO} & W_{0,1}^{VO} \\ W_{1,0}^{VO} & W_{1,1}^{VO} \end{pmatrix}, \end{aligned}$$

长度会导致负载不平衡（“气泡”），从而浪费计算资源。管道并行性 [44, 45] 将模型划分为连续的层块，每个层块放置在不同的设备上。中间激活和梯度在阶段之间进行通信，以完成前向和后向传递，从而减少跨节点流量。这种分段会跨设备重叠计算以提高吞吐量，但管道气泡仍然可能导致某些设备闲置。张量并行性 [46, 47, 48] 沿行或列维度分割线性层，跨设备分片张量并通过集体通信执行分布式矩阵-矩阵乘法。它在 GPU 通过 NVLink 完全互连的系统上实现了最佳性能。TPLA 利用了 TP 的优势，同时解决了 MLA 在 TP 下无法减少 KV 缓存的问题。长序列会增加中间激活的内存占用；序列并行性 [49] 通过跨设备复制模型并沿序列维度分割输入来缓解这种情况，以便每个设备仅处理一个子序列。预填充/解码分离 [50, 51, 52] 改进了 LLM 推理的序列并行性：预填充阶段是计算密集型的，因此受计算限制，而解码阶段的每个令牌计算量较低，但内存访问频繁，并且受内存带宽限制。为了匹配这些特征，两个阶段使用不同的机器数量和架构来改善延迟和吞吐量。在 TPLA 中，我们进一步跨阶段采用不同的模型结构——预填充期间的 MLA 可以在减少计算（改善延迟）的同时保持准确性，解码期间的 TPLA 可以减少内存流量并提高吞吐量。

### 3 初步

#### 3.1 多头潜在注意力

MLA 旨在通过压缩键值 (KV) 缓存来减少内存带宽开销。具体来说，多头键和值被压缩为维度  $4d_h$  的单个低秩潜在表示，表示为  $\mathbf{c}^{\text{KV}}$ 。MLA 没有从这个潜在表示中重建全尺寸的键和值，而是采用了更有效的解码策略。通过隔离旋转位置嵌入 (RoPE) 操作，上投影矩阵可以被吸收到查询激活中，产生  $Q$ 。类似地，值投影被吸收到输出投影矩阵中，从而产生  $W^{VO}$  (参见第 3.3 节)。这允许在  $Q$  和标准化潜在缓存  $\hat{\mathbf{c}}^{\text{KV}}$  之间进行直接注意计算，然后通过  $W^{VO}$  进行投影以产生最终输出  $\tilde{O}$ 。为了简单起见，我们在最初的描述中省略了 RoPE 组件。核心计算如下：

$$\begin{aligned} \mathbf{c}^{\text{KV}} &\in \mathbb{R}^{B \times L \times 4d_h}, \quad \hat{\mathbf{c}}^{\text{KV}} = \text{RMSNorm}(\mathbf{c}^{\text{KV}}) \in \mathbb{R}^{B \times L \times 4d_h}, \\ Q &\in \mathbb{R}^{B \times 1 \times h_q \times 4d_h}, \quad W^{VO} \in \mathbb{R}^{(h_q \cdot 4d_h) \times D}, \\ O &= \text{softmax}\left(\frac{Q(\hat{\mathbf{c}}^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 4d_h}, \\ \tilde{O} &= O W^{VO} \in \mathbb{R}^{B \times 1 \times D}. \end{aligned} \tag{1}$$

$$\tag{2}$$

#### 3.2 分组潜在注意力

在张量并行解码期间，MLA 在每个设备上复制其单个潜在头，从而导致所有设备上的高 KV 缓存负载。GLA 通过对潜在 KV 缓存本身进行分区来避免这种复制。考虑双向张量并行配置。潜在的 KV 缓存分为两个分片  $\mathbf{c}_0^{\text{KV}}$  和  $\mathbf{c}_1^{\text{KV}}$ ，每个分片分配给一个 GPU。同时，注意力头  $h_q$  被分组，使得吸收的查询投影矩阵  $Q$  和输出投影矩阵  $W^{VO}$  沿着头维度 ( $h_q$ ) 和特征维度 ( $4d_h$ ) 划分，产生四组。因此，GPU 0 在  $(\mathbf{c}_0^{\text{KV}}, Q_{0,0}, W_{0,0}^{VO})$  上运行，而 GPU 1 在  $(\mathbf{c}_1^{\text{KV}}, Q_{1,1}, W_{1,1}^{VO})$  上运行。每个 GPU 独立计算其局部注意力输出，分别表示为  $\tilde{O}_0$  和  $\tilde{O}_1$ 。最终输出是通过 AllReduce 操作获得的，该操作对跨设备的本地输出求和。

$$\begin{aligned} \mathbf{c}_0^{\text{KV}}, \mathbf{c}_1^{\text{KV}} &\in \mathbb{R}^{B \times L \times 2d_h}, \quad \begin{cases} \hat{\mathbf{c}}_0^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_0^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \\ \hat{\mathbf{c}}_1^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_1^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \end{cases} \\ Q_{i,j \in \{0,1\}} &\in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \quad \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix} = Q, \\ W_{i,j \in \{0,1\}}^{VO} &\in \mathbb{R}^{\left(\frac{h_q}{2} \cdot 2d_h\right) \times D}, \quad W^{VO} = \begin{pmatrix} W_{0,0}^{VO} & W_{0,1}^{VO} \\ W_{1,0}^{VO} & W_{1,1}^{VO} \end{pmatrix}, \end{aligned}$$

$$\begin{aligned}
O_0 &= \text{softmax}\left(\frac{Q_{0,0} (\hat{\mathbf{c}}_0^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_0^{\text{KV}} \in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \\
O_1 &= \text{softmax}\left(\frac{Q_{1,1} (\hat{\mathbf{c}}_1^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_1^{\text{KV}} \in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \\
\tilde{O}_0 &= O_0 W_{0,0}^{VO} \in \mathbb{R}^{B \times 1 \times D}, \quad \tilde{O}_1 = O_1 W_{1,1}^{VO} \in \mathbb{R}^{B \times 1 \times D}, \\
O &= \text{AllReduce}(\tilde{O}_0 + \tilde{O}_1) \in \mathbb{R}^{B \times 1 \times D}.
\end{aligned} \tag{3}$$

### 3.3 Matrix Absorption

Considering we apply orthogonal transformations  $U$  to reparameterize weight matrices, which involves matrix absorption. To make this process intuitive, we here present the complete calculation pipeline of MLA and show how the absorbed matrices from Section 3.1 are derived.

As stated in Section 3.1, MLA saves KV cache by multiplying the low-rank compress matrix  $W^{DKV} \in \mathbb{R}^{D \times 4d_h}$  with the input sequence  $X \in \mathbb{R}^{B \times L \times D}$  to obtain low-rank latent features  $\mathbf{c}^{\text{KV}}$ . Then, it uses the matrices  $W^{UK}, W^{UV} \in \mathbb{R}^{4d_h \times (h_q \cdot d_h)}$  to derive the full-heads key  $\mathbf{k}$  and value  $\mathbf{v}$ . Additionally, MLA also can decompose  $W^Q \in \mathbb{R}^{D \times (h_q \cdot d_h)}$  to  $W^{DQ} \in \mathbb{R}^{D \times r_q}$  and  $W^{UQ} \in \mathbb{R}^{r_q \times (h_q \cdot d_h)}$ , which reduces the activation memory during training. For positional embedding, MLA uses a decoupled RoPE strategy that uses additional multi-head queries  $\mathbf{q}^{\text{PE}}$  and a shared key  $\mathbf{k}^{\text{PE}}$ , generated by  $W^{QR} \in \mathbb{R}^{r_q \times (h_q \cdot d_r)}$  and  $W^{KR} \in \mathbb{R}^{D \times d_r}$ , to carry the rotary positional embeddings. The final attention output  $\tilde{O}$  is computed by separately combining the non-positional part ( $\mathbf{q} \mathbf{k}^\top$ ) and positional part ( $\mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top$ ), followed by projection with  $W^O \in \mathbb{R}^{(h_q \cdot d_h) \times D}$ .

$$\begin{aligned}
\mathbf{c}^{\text{KV}} &= X W^{DKV}, \quad \mathbf{c}^Q = X W^{DQ}, \quad \hat{\mathbf{c}}^{\text{KV}} = \text{RMSNorm}(\mathbf{c}^{\text{KV}}), \\
\mathbf{q} &= \mathbf{c}^Q W^{UQ}, \quad \mathbf{k} = \hat{\mathbf{c}}^{\text{KV}} W^{UK}, \quad \mathbf{v} = W^{UV} \hat{\mathbf{c}}^{\text{KV}}, \\
\mathbf{q}^{\text{PE}} &= \text{RoPE}(\mathbf{c}^Q W^{QR}), \quad \mathbf{k}^{\text{PE}} = \text{RoPE}(X W^{KR}), \\
O &= \text{softmax}\left(\frac{\mathbf{q} \mathbf{k}^\top + \mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top}{\sqrt{d_h + d_r}}\right) \mathbf{v}, \quad \tilde{O} = O W^O.
\end{aligned} \tag{4}$$

In Equation 4, the RoPE component is explicitly isolated, allowing us to restructure the attention computation using associativity of matrix multiplication. For clarity, we can temporarily omit positional encoding components and the scaling factor.

$$\begin{aligned}
O W^O &= \text{softmax}(\mathbf{q} \mathbf{k}^\top) \mathbf{v} W^O \\
&= \text{softmax}(\mathbf{q} (\hat{\mathbf{c}}^{\text{KV}} W^{UK})^\top) \hat{\mathbf{c}}^{\text{KV}} W^{UV} W^O \\
&= \text{softmax}(Q (\hat{\mathbf{c}}^{\text{KV}})^\top) \hat{\mathbf{c}}^{\text{KV}} W^{VO}.
\end{aligned} \tag{5}$$

Here, the matrix  $W^{UK}$  can be absorbed into  $\mathbf{q}$  to derive  $Q$  in Equation 2. Similarly, the matrix  $W^{UV}$  can be absorbed into  $W^O$ . In practice, however,  $W^{UV}$  is typically not absorbed into  $W^O$  to avoid generating an impractically large matrix.

$$\begin{aligned}
O_0 &= \text{softmax}\left(\frac{Q_{0,0} (\hat{\mathbf{c}}_0^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_0^{\text{KV}} \in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \\
O_1 &= \text{softmax}\left(\frac{Q_{1,1} (\hat{\mathbf{c}}_1^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_1^{\text{KV}} \in \mathbb{R}^{B \times 1 \times \frac{h_q}{2} \times 2d_h}, \\
\tilde{O}_0 &= O_0 W_{0,0}^{VO} \in \mathbb{R}^{B \times 1 \times D}, \quad \tilde{O}_1 = O_1 W_{1,1}^{VO} \in \mathbb{R}^{B \times 1 \times D}, \\
O &= \text{AllReduce}(\tilde{O}_0 + \tilde{O}_1) \in \mathbb{R}^{B \times 1 \times D}.
\end{aligned} \tag{3}$$

### 3.3 矩阵吸收

考虑到我们应用正交变换  $U$  来重新参数化权重矩阵，这涉及矩阵吸收。为了使这个过程直观，我们在这里展示了 MLA 的完整计算流程，并展示了如何导出第 3.1 节中的吸收矩阵。

如3.1节所述，MLA通过将低秩压缩矩阵  $W^{DKV} \in \mathbb{R}^{D \times 4d_h}$  与输入序列  $X \in \mathbb{R}^{B \times L \times D}$  相乘以获得低秩潜在特征  $\mathbf{c}^{\text{KV}}$  来节省KV缓存。然后，它使用矩阵  $W^{UK}, W^{UV} \in \mathbb{R}^{4d_h \times (h_q \cdot d_h)}$  导出全头键  $\mathbf{k}$  和值  $\mathbf{v}$ 。此外，MLA还可以将  $W^Q \in \mathbb{R}^{D \times (h_q \cdot d_h)}$  分解为  $W^{DQ} \in \mathbb{R}^{D \times r_q}$  和  $W^{UQ} \in \mathbb{R}^{r_q \times (h_q \cdot d_h)}$ ，从而减少训练期间的激活记忆。对于位置嵌入，MLA 使用解耦的 RoPE 策略，该策略使用额外的多头查询  $\mathbf{q}^{\text{PE}}$  和由  $W^{QR} \in \mathbb{R}^{r_q \times (h_q \cdot d_r)}$  和  $W^{KR} \in \mathbb{R}^{D \times d_r}$  生成的共享密钥  $\mathbf{k}^{\text{PE}}$  来承载旋转位置嵌入。最终的注意力输出  $\tilde{O}$  是通过分别组合非位置部分 ( $\mathbf{q} \mathbf{k}^\top$ ) 和位置部分 ( $\mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top$ ) 来计算的，然后用  $W^O \in \mathbb{R}^{(h_q \cdot d_h) \times D}$  进行投影。

$$\begin{aligned}
\mathbf{c}^{\text{KV}} &= X W^{DKV}, \quad \mathbf{c}^Q = X W^{DQ}, \quad \hat{\mathbf{c}}^{\text{KV}} = \text{RMSNorm}(\mathbf{c}^{\text{KV}}), \\
\mathbf{q} &= \mathbf{c}^Q W^{UQ}, \quad \mathbf{k} = \hat{\mathbf{c}}^{\text{KV}} W^{UK}, \quad \mathbf{v} = W^{UV} \hat{\mathbf{c}}^{\text{KV}}, \\
\mathbf{q}^{\text{PE}} &= \text{RoPE}(\mathbf{c}^Q W^{QR}), \quad \mathbf{k}^{\text{PE}} = \text{RoPE}(X W^{KR}), \\
O &= \text{softmax}\left(\frac{\mathbf{q} \mathbf{k}^\top + \mathbf{q}^{\text{PE}} (\mathbf{k}^{\text{PE}})^\top}{\sqrt{d_h + d_r}}\right) \mathbf{v}, \quad \tilde{O} = O W^O.
\end{aligned} \tag{4}$$

在等式 4 中，RoPE 组件被明确隔离，允许我们使用矩阵乘法的关联性来重构注意力计算。为了清楚起见，我们可以暂时省略位置编码组件和缩放因子。

$$\begin{aligned}
O W^O &= \text{softmax}(\mathbf{q} \mathbf{k}^\top) \mathbf{v} W^O \\
&= \text{softmax}(\mathbf{q} (\hat{\mathbf{c}}^{\text{KV}} W^{UK})^\top) \hat{\mathbf{c}}^{\text{KV}} W^{UV} W^O \\
&= \text{softmax}(Q (\hat{\mathbf{c}}^{\text{KV}})^\top) \hat{\mathbf{c}}^{\text{KV}} W^{VO}.
\end{aligned} \tag{5}$$

这里，可以将矩阵  $W^{UK}$  吸收到  $\mathbf{q}$  中以导出等式2中的  $Q$ 。类似地，可以将矩阵  $W^{UK}$  吸收到  $W^O$  中。然而，实际上， $W^{UV}$  通常不会被吸收到  $W^O$  中，以避免生成不切实际的大矩阵。

## 4 Tensor Parallel Latent Attention (TPLA)

Motivated by the hardware efficiency of GLA, we retain its core principle of distributing latent KV across GPUs to mitigate memory wastage and communication overload. However, directly translating an existing MLA-based model to GLA incurs a significant performance penalty, as shown in Figure 2. This degradation stems from a key limitation in standard GLA: the latent vector within each group only accesses half of the query heads, restricting the model’s expressive power and leading to suboptimal performance. Moreover, training a new GLA model from scratch requires a substantial cost. To address this, we further propose Tensor-Parallel Latent Attention (TPLA). Unlike standard GLA, TPLA partitions latent vectors into two groups while preserving full query heads visibility. Specifically,

$$\begin{aligned}
\hat{\mathbf{c}}_0^{\text{KV}}, \hat{\mathbf{c}}_1^{\text{KV}} &\in \mathbb{R}^{B \times L \times 2d_h}, \begin{cases} \hat{\mathbf{c}}_0^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_0^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \\ \hat{\mathbf{c}}_1^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_1^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \end{cases} \\
Q_0, Q_1 &\in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \quad (Q_0, Q_1) = Q, \\
W_0^{VO}, W_1^{VO} &\in \mathbb{R}^{(h_q \cdot 2d_h) \times D}, \quad (W_0^{VO}, W_1^{VO}) = W^{VO}, \\
O_0 &= \text{softmax}\left(\frac{Q_0 (\hat{\mathbf{c}}_0^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_0^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \\
O_1 &= \text{softmax}\left(\frac{Q_1 (\hat{\mathbf{c}}_1^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_1^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \\
\tilde{O}_0 &= O_0 W_0^{VO} \in \mathbb{R}^{B \times 1 \times D}, \quad \tilde{O}_1 = O_1 W_1^{VO} \in \mathbb{R}^{B \times 1 \times D}, \\
O &= \text{AllReduce}(\tilde{O}_0 + \tilde{O}_1) \in \mathbb{R}^{B \times 1 \times D}.
\end{aligned} \tag{6}$$

$$\tag{7}$$

This design ensures each latent vector attends to all query heads, mitigating the most performance loss. The residual performance loss now stems exclusively from tensor-parallel partitioning effects in RMSNorm and softmax operations. Through carefully designed mathematical reparameterization, TPLA can restore near-MLA performance. For illustration, we consider the case where the tensor-parallel degree of latent attention is 2, though the approach naturally scales to higher degrees.

### 4.1 RMSNorm Slicing

In MLA-like models, the “kv\_a\_layernorm” module normalizes input vectors using the Root Mean Square (RMS) value. Given an input vector  $\mathbf{x} \in \mathbb{R}^d$  (e.g.,  $d = 4d_h$ ), the RMSNorm is computed as:

$$\begin{aligned}
\text{RMS}(\mathbf{x}) &= \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \\
&= \sqrt{\frac{1}{d} \|\mathbf{x}\|_2^2 + \epsilon},
\end{aligned} \tag{8}$$

$$\begin{aligned}
\text{RMSNorm}(\gamma, \mathbf{x}) &= \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma \\
&= \text{RMSNorm}(\mathbf{1}, \mathbf{x}) \odot \gamma,
\end{aligned} \tag{9}$$

where  $\epsilon$  is a small constant for numerical stability;  $\gamma \in \mathbb{R}^d$  is a learned scaling parameter and  $\odot$  denotes element-wise multiplication.

However, we face the following challenge when applying this to tensor-parallel processing of latent attention: When input latent vector  $\mathbf{x} \in \mathbb{R}^d$  is split into two partitions,  $\mathbf{x}^{(0)} \in \mathbb{R}^{d/2}$  and  $\mathbf{x}^{(1)} \in \mathbb{R}^{d/2}$ , across different devices, the RMS computation on each local device uses only half the original dimension ( $d/2$ ), while the true normalization requires the full RMS( $\mathbf{x}$ ) over dimension  $d$ .

To resolve this discrepancy, we introduce an orthogonal transformation  $U \in \mathbb{R}^{d \times d}$  ( $U U^\top = \mathbf{I}$ ) to reparameterize this module. Before introducing the conditions that this transformation  $U$  need satisfy, we first establish that RMSNorm can, in principle, be realized in a mathematically equivalent form under any orthogonal transformation.

**Proposition 1.**

$$\text{RMSNorm}(\mathbf{1}, \mathbf{c}) = \text{RMSNorm}(\mathbf{1}, \mathbf{c} U) U^\top \tag{10}$$

#### 4 张量并行潜在注意力 (TPLA)

受 GLA 硬件效率的推动, 我们保留了其在 GPU 之间分配潜在 KV 的核心原则, 以减少内存浪费和通不过载。然而, 直接将现有的基于 MLA 的模型转换为 GLA 会导致显著的性能损失, 如图 2 所示。这种降级源于标准 GLA 中的一个关键限制: 每组内的潜在向量仅访问一半的查询头, 限制了模型的表达能力并导致性能不佳。此外, 从头开始训练一个新的 GLA 模型需要大量成本。为了解决这个问题, 我们进一步提出张量并行潜在注意力 (TPLA)。与标准 GLA 不同, TPLA 将潜在向量划分为两组, 同时保留完整的查询头可见性。具体来说,

$$\begin{aligned}
 \mathbf{c}_0^{\text{KV}}, \mathbf{c}_1^{\text{KV}} &\in \mathbb{R}^{B \times L \times 2d_h}, \begin{cases} \hat{\mathbf{c}}_0^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_0^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \\ \hat{\mathbf{c}}_1^{\text{KV}} = \text{RMSNorm}(\mathbf{c}_1^{\text{KV}}) \in \mathbb{R}^{B \times L \times 2d_h}, \end{cases} \\
 Q_0, Q_1 &\in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \quad (Q_0, Q_1) = Q, \\
 W_0^{VO}, W_1^{VO} &\in \mathbb{R}^{(h_q \cdot 2d_h) \times D}, \quad (W_0^{VO}, W_1^{VO}) = W^{VO}, \\
 O_0 &= \text{softmax}\left(\frac{Q_0 (\hat{\mathbf{c}}_0^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_0^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \\
 O_1 &= \text{softmax}\left(\frac{Q_1 (\hat{\mathbf{c}}_1^{\text{KV}})^\top}{\sqrt{d_h}}\right) \hat{\mathbf{c}}_1^{\text{KV}} \in \mathbb{R}^{B \times 1 \times h_q \times 2d_h}, \tag{6} \\
 \tilde{O}_0 &= O_0 W_0^{VO} \in \mathbb{R}^{B \times 1 \times D}, \quad \tilde{O}_1 = O_1 W_1^{VO} \in \mathbb{R}^{B \times 1 \times D}, \\
 O &= \text{AllReduce}(\tilde{O}_0 + \tilde{O}_1) \in \mathbb{R}^{B \times 1 \times D}. \tag{7}
 \end{aligned}$$

这种设计确保每个潜在向量关注所有查询头, 从而最大限度地减少性能损失。现在, 残余性能损失完全源于 RMSNorm 和 softmax 运算中的张量并行划分效应。通过精心设计的数学重新参数化, TPLA 可以恢复接近 MLA 的性能。为了便于说明, 我们考虑潜在注意力的张量并行度为 2 的情况, 尽管该方法自然可以扩展到更高的程度。

##### 4.1 RMSNorm 切片

在类似 MLA 的模型中, “kv\_a\_layernorm” 模块使用均方根 (RMS) 值对输入向量进行归一化。给定输入向量  $\mathbf{x} \in \mathbb{R}^d$  (例如  $d = 4d_h$ ), RMSNorm 计算如下:

$$\begin{aligned}
 \text{RMS}(\mathbf{x}) &= \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \\
 &= \sqrt{\frac{1}{d} \|\mathbf{x}\|_2^2 + \epsilon}, \tag{8}
 \end{aligned}$$

$$\begin{aligned}
 \text{RMSNorm}(\gamma, \mathbf{x}) &= \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma \\
 &= \text{RMSNorm}(\mathbf{1}, \mathbf{x}) \odot \gamma, \tag{9}
 \end{aligned}$$

其中  $\epsilon$  是一个用于数值稳定性的小常数;  $\gamma \in \mathbb{R}^d$  是学习的缩放参数,  $\odot$  表示逐元素乘法。

然而, 将其应用于潜在注意力的张量并行处理时, 我们面临以下挑战: 当输入潜在向量  $\mathbf{x} \in \mathbb{R}^d$  跨不同设备分为两个分区  $\mathbf{x}^{(0)} \in \mathbb{R}^{d/2}$  和  $\mathbf{x}^{(1)} \in \mathbb{R}^{d/2}$  时, 每个本地设备上的 RMS 计算仅使用原始维度的一半 ( $d/2$ ), 而真正的归一化需要维度  $d$  上的完整  $\text{RMS}(\mathbf{x})$ 。

为了解决这个差异, 我们引入了正交变换  $U \in \mathbb{R}^{d \times d}$  ( $U U^\top = \mathbf{I}$ ) 来重新参数化该模块。在介绍这个变换  $U$  需要满足的条件之前, 我们首先确定 RMSNorm 原则上可以在任何正交变换下以数学上等价的形式实现。

命题1。

$$\text{RMSNorm}(\mathbf{1}, \mathbf{c}) = \text{RMSNorm}(\mathbf{1}, \mathbf{c} U) U^\top \tag{10}$$

*Proof.* we first represent the RMSNorm process as matrix multiplication. Let  $\mathbf{c} \in \mathbb{R}^{L \times d}$  be the input latent vector (for simplicity, we omit the batch size), we obtain:

$$\text{RMSNorm}(\gamma, \mathbf{c}) = \text{RMSNorm}(\mathbf{1}, \mathbf{c}) W_\gamma, \quad (11)$$

$$= D_c \mathbf{c} W_\gamma, \quad (12)$$

where  $D_c$  is a diagonal matrix of size  $L \times L$  with the reciprocal of the RMS values on the diagonal and  $W_\gamma$  is also a diagonal matrix of size  $d \times d$  with each learnable scaling parameter:

$$D_c = \text{diag} \left( \frac{1}{\text{RMS}(c_1)}, \frac{1}{\text{RMS}(c_2)}, \dots, \frac{1}{\text{RMS}(c_L)} \right), \quad (13)$$

$$W_\gamma = \text{diag} (\gamma_1, \gamma_2, \dots, \gamma_d). \quad (14)$$

Since the orthogonal transformation preserves the norm ( $\|\mathbf{c} U\|_2^2 = \|\mathbf{c}\|_2^2$ ), we easily have  $\text{RMS}(\mathbf{c}) = \text{RMS}(\mathbf{c} U)$ , i.e.,  $D_{cU} = D_c$ . Thus, we can have:

$$\text{RMSNorm}(\gamma, \mathbf{c} U) U^\top = D_c \mathbf{c} U W_\gamma U^\top. \quad (15)$$

Matrix multiplication does not satisfy the commutative property. Therefore, when and only when  $W_\gamma = \mathbf{I}$ , we can further prove:

$$\begin{aligned} \text{RMSNorm}(\mathbf{1}, \mathbf{c} U) U^\top &= D_c \mathbf{c} U \mathbf{I} U^\top \\ &= D_c \mathbf{c} \\ &= \text{RMSNorm}(\mathbf{1}, \mathbf{c}). \end{aligned} \quad (16)$$

□

Give by Equation 5, Equation 11 and Proposition 1, we can absorb  $W_\gamma$  into up-projection matrix  $W^{UKV} = (W^{UK}, W^{UV})$  to achieve the  $\gamma = \mathbf{1}$ , ensuring the orthogonal transformations  $U$  to  $\mathbf{c}$  with keeping the RMSNorm value no change. In addition, the  $U^\top$  can be further absorbed into  $W^{UKV}$ ;  $U$  can be absorbed into  $W^{DKV}$ , yielding the reparameterized weight matrix:

$$W_{new}^{UKV} = U^\top W_\gamma W^{UKV}, \quad W_{new}^{DKV} = W^{DKV} U. \quad (17)$$

We have proved that any transformation  $U$  can ensure the equivalence of RMSNorm. Now we will define some conditions that serve as the computational basis for  $U$ , deferring the specific calculation method to a later section.

**Condition 1** (RMSNorm Slicing Condition).

$$\alpha \|(\mathbf{c} U)_0\|_2^2 \approx \beta \|(\mathbf{c} U)_1\|_2^2 \approx \|\mathbf{c} U\|_2^2 = \|\mathbf{c}\|_2^2. \quad (18)$$

Here,  $\alpha$  and  $\beta$  are fixed constants, invariant to changes in the input data distribution (How to calculate their specific values is detailed in Section 4.3).  $(\mathbf{c} U)_1$  and  $(\mathbf{c} U)_2$  are the two partitions of the transformed  $\mathbf{c}$  split across devices. By satisfying this, the new RMS values computed from the two partitions are proportional to the global value, thereby providing an accurate approximation of the global RMSNorm:

$$\begin{aligned} \text{RMS}(\mathbf{c}) &= \sqrt{\frac{1}{d} \|\mathbf{c}\|_2^2 + \epsilon} \\ &\approx \sqrt{\frac{\alpha}{d} \|(\mathbf{c} U)_0\|_2^2 + \epsilon} \\ &\approx \sqrt{\frac{\alpha}{2}} \text{RMS}((\mathbf{c} U)_0) \approx \sqrt{\frac{\beta}{2}} \text{RMS}((\mathbf{c} U)_1). \end{aligned} \quad (19)$$

Thus, we can compute RMSNorm in a tensor-parallel manner while maintaining the integrity of the normalization process.

## 4.2 Softmax Slicing

In common tensor-parallel techniques, matrices are typically split across devices to perform either row or column parallelism. For our TPLA attention score computation, row parallelism is employed, where the weight matrix  $A$  is split across devices according to its rows. To ensure a valid matrix multiplication, the input matrix  $X$  is correspondingly partitioned column-wise into  $X_1$  and  $X_2$ , such that

$$XA = (X_1 \quad X_2) \begin{pmatrix} A_1 \\ A_2 \end{pmatrix} = X_1 \cdot A_1 + X_2 \cdot A_2 = Y_1 + Y_2 = Y \quad (20)$$

*Proof.* 我们首先将 RMSNorm 过程表示为矩阵乘法。令  $\mathbf{c} \in \mathbb{R}^{L \times d}$  为输入潜在向量（为简单起见，我们省略批量大小），我们得到：

$$\text{RMSNorm}(\gamma, \mathbf{c}) = \text{RMSNorm}(\mathbf{1}, \mathbf{c}) W_\gamma, \quad (11)$$

$$= D_c \mathbf{c} W_\gamma, \quad (12)$$

其中  $D_c$  是大小为  $L \times L$  的对角矩阵，对角线上具有 RMS 值的倒数，而  $W_\gamma$  也是大小为  $d \times d$  的对角矩阵，其中每个可学习缩放参数：

$$D_c = \text{diag} \left( \frac{1}{\text{RMS}(c_1)}, \frac{1}{\text{RMS}(c_2)}, \dots, \frac{1}{\text{RMS}(c_L)} \right), \quad (13)$$

$$W_\gamma = \text{diag}(\gamma_1, \gamma_2, \dots, \gamma_d). \quad (14)$$

由于正交变换保留了范数 ( $\|\mathbf{c}U\|_2^2 = \|\mathbf{c}\|_2^2$ )，因此我们很容易得到  $\text{RMS}(\mathbf{c}) = \text{RMS}(\mathbf{c}U)$ ，即  $D_{cU} = D_c$ 。因此，我们可以有：

$$\text{RMSNorm}(\gamma, \mathbf{c}U) U^\top = D_c \mathbf{c}U W_\gamma U^\top. \quad (15)$$

矩阵乘法不满足交换律。因此，当且仅当  $W_\gamma = \mathbf{I}$  时，我们可以进一步证明：

$$\begin{aligned} \text{RMSNorm}(\mathbf{1}, \mathbf{c}U) U^\top &= D_c \mathbf{c}U \mathbf{I} U^\top \\ &= D_c \mathbf{c} \\ &= \text{RMSNorm}(\mathbf{1}, \mathbf{c}). \end{aligned} \quad (16)$$

□

由式5、式11和命题1可知，我们可以将  $W_\gamma$  吸纳到上投影矩阵  $W^{UKV} = (W^{UK}, W^{UV})$  中，得到  $\gamma = \mathbf{1}$ ，保证  $U$  到  $\mathbf{c}$  的正交变换，同时保持 RMSNorm 值不变。另外， $U^\top$  可以进一步被吸收为  $W^{UKV}$ ； $U$  可以被吸收到  $W^{DKV}$  中，产生重新参数化的权重矩阵：

$$W_{new}^{UKV} = U^\top W_\gamma W^{UKV}, \quad W_{new}^{DKV} = W^{DKV} U. \quad (17)$$

我们已经证明任何变换  $U$  都可以保证 RMSNorm 的等价性。现在我们定义一些条件作为  $U$  的计算基础，具体的计算方法留到后面的章节。

条件 1 (RMSNorm 切片条件)。

$$\alpha \|(\mathbf{c}U)_0\|_2^2 \approx \beta \|(\mathbf{c}U)_1\|_2^2 \approx \|\mathbf{c}U\|_2^2 = \|\mathbf{c}\|_2^2. \quad (18)$$

这里， $\alpha$  和  $\beta$  是固定常量，不随输入数据分布的变化而变化（如何计算它们的具体值在 4.3 节中有详细介绍）。 $(\mathbf{c}U)_1$  和  $(\mathbf{c}U)_2$  是跨设备分割的转换后的  $\mathbf{c}$  的两个分区。通过满足这一点，从两个分区计算出的新 RMS 值与全局值成比例，从而提供全局 RMSNorm 的精确近似值：

$$\begin{aligned} \text{RMS}(\mathbf{c}) &= \sqrt{\frac{1}{d} \|\mathbf{c}\|_2^2 + \epsilon} \\ &\approx \sqrt{\frac{\alpha}{d} \|(\mathbf{c}U)_0\|_2^2 + \epsilon} \\ &\approx \sqrt{\frac{\alpha}{2}} \text{RMS}((\mathbf{c}U)_0) \approx \sqrt{\frac{\beta}{2}} \text{RMS}((\mathbf{c}U)_1). \end{aligned} \quad (19)$$

因此，我们可以以张量并行的方式计算 RMSNorm，同时保持归一化过程的完整性。

## 4.2 Softmax 切片

在常见的张量并行技术中，矩阵通常跨设备分割以执行行或列并行。对于我们的 TPLA 注意力分数计算，采用行并行，其中权重矩阵  $A$  根据其行在设备之间分割。为了确保有效的矩阵乘法，输入矩阵  $X$  相应地按列划分为  $X_1$  和  $X_2$ ，使得

$$XA = (X_1 \quad X_2) \begin{pmatrix} A_1 \\ A_2 \end{pmatrix} = X_1 \cdot A_1 + X_2 \cdot A_2 = Y_1 + Y_2 = Y \quad (20)$$

where  $X_1$  and  $A_1$  are computed on GPU 0 to produce  $Y_1$ , and  $X_2$  and  $A_2$  are computed on GPU 1 to produce  $Y_2$ . The final output  $Y$  is then all-reduced by summing  $Y_1$  and  $Y_2$ .

In the context of softmax computation (Equation 4), TPLA partitions  $\mathbf{c}^{\text{KV}}$  and ensures that the computation of positional components remains unaffected. Specifically, the shard of the key positional embedding  $\mathbf{k}^{\text{PE}}$  must be replicated across devices so that the local positional values remain consistent with the global values. As for non-positional parts ( $Q(\hat{\mathbf{c}}^{\text{KV}})^\top$ ), the latent vectors  $\mathbf{c}$  are split into two devices, and each device performs only its local computation, i.e., GPU 0 computes  $Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top$ , while GPU 1 computes  $Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top$ . However, in most cases,

$$\begin{aligned} & \text{softmax}(Q(\hat{\mathbf{c}}^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \\ &= \text{softmax}(Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top + Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \\ &\neq \text{softmax}(Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \quad [\text{GPU 0}] \\ &\neq \text{softmax}(Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \quad [\text{GPU 1}] \end{aligned}$$

Thus, the challenge of TPLA is how to ensure the global value of  $Q\mathbf{c}^{\text{KV}}^\top$  can be approximated from local computations. We first show that applying any orthogonal transformation  $U$  does not alter the equivalence of the original softmax output. Based on Equation 5, we easily have:

$$\begin{aligned} Q(\hat{\mathbf{c}}^{\text{KV}})^\top &= QU(\hat{\mathbf{c}}^{\text{KV}}U)^\top = \mathbf{q}(U^\top W^{UK})^\top (\hat{\mathbf{c}}^{\text{KV}}U)^\top \\ &= Q'(\hat{\mathbf{c}}^{\text{KV}}U)^\top \end{aligned} \quad (21)$$

Analogous to Section 4.1, by absorbing  $U$  into  $W^{UKV}$  and  $W^{DKV}$  (equivalently, into  $Q$  to obtain  $Q'$ ), we can impose an orthogonal transformation  $U$ , which preserves the original softmax computation and must satisfy:

**Condition 2** (Softmax Slicing Condition).

$$Q'(\hat{\mathbf{c}}^{\text{KV}}U)^\top \approx \mu Q'_0(\hat{\mathbf{c}}^{\text{KV}}U)_0^\top \approx \nu Q'_1(\hat{\mathbf{c}}^{\text{KV}}U)_1^\top \quad (22)$$

Accordingly, by determining the coefficients  $\mu$  and  $\nu$ , each device can perform its local computation and scale by the factor, thereby approximating the global value.

### 4.3 Reparameterization Methods

From the derivation above, we need to find one orthogonal transformation matrix  $U$  applied to projection weights, ensuring that the transformation satisfies Condition 1 and Condition 2 — local computations can accurately approximate the global RMSNorm and softmax values. To achieve this, we explore two potential methods: Hadamard Matrix Transformation and Principal Component Analysis (PCA).

#### 4.3.1 Hadamard Matrix Transformation

Hadamard matrix is a special orthogonal matrix where each entry is either +1 or -1. It operates by balancing the numbers, thereby reducing extreme numerical deviations and promoting a more uniform distribution of data. In practice, we typically use the function `scipy.linalg.hadamard(d)` to generate a Sylvester-type Hadamard matrix (also known as a Walsh-Hadamard matrix)  $H_d \in \mathbb{R}^{d \times d}$ , constructed using a deterministic recursive rule:

$$H_{2n} = \begin{pmatrix} H_n & H_n \\ H_n & -H_n \end{pmatrix}, \quad H_1 = (1). \quad (23)$$

To increase robustness, a random diagonal matrix  $D$ , with entries drawn from  $\pm 1$  is multiplied with  $H_d$ , thereby breaking deterministic structure while preserving orthogonality. Since  $H_d H_d^\top = d \cdot \mathbf{I}$ , orthonormality is achieved by scaling  $H_d$  by  $\frac{1}{\sqrt{d}}$ , ensuring that normalization values are preserved.

Take an illustrative example. Consider a 4-dimensional vector  $\mathbf{c} = (100, 0, 0, 0)$  and the  $4 \times 4$  Hadamard matrix  $H_4$ . The transformed vector  $\mathbf{c}' = \mathbf{c}H_4$  is:

$$\mathbf{c}' = (100, 0, 0, 0) \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} = (50, 50, 50, 50). \quad (24)$$

其中  $X_1$  和  $A_1$  在 GPU 0 上计算以生成  $Y_1$ ，而  $X_2$  和  $A_2$  在 GPU 1 上计算以生成  $Y_2$ 。然后通过对  $Y_1$  和  $Y_2$  求和来对最终输出  $Y$  进行全约简。

在 softmax 计算（公式 4）的背景下，TPLA 划分  $\mathbf{c}^{\text{KV}}$  并确保位置分量的计算不受影响。具体来说，关键位置嵌入  $\mathbf{k}^{\text{PE}}$  的分片必须跨设备复制，以便本地位置值与全局值保持一致。对于非位置部分  $(Q(\hat{\mathbf{c}}^{\text{KV}})^\top)$ ，潜在向量  $\mathbf{c}$  被分成两个设备，每个设备仅执行其本地计算，即 GPU 0 计算  $Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top$ ，而 GPU 1 计算  $Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top$ 。然而，在大多数情况下，

$$\begin{aligned} & \text{softmax}(Q(\hat{\mathbf{c}}^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \\ &= \text{softmax}(Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top + Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \\ &\neq \text{softmax}(Q_0(\hat{\mathbf{c}}_0^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \quad [\text{GPU 0}] \\ &\neq \text{softmax}(Q_1(\hat{\mathbf{c}}_1^{\text{KV}})^\top + \mathbf{q}^{\text{PE}}(\mathbf{k}^{\text{PE}})^\top) \quad [\text{GPU 1}] \end{aligned}$$

因此，TPLA 的挑战是如何确保  $Q(\mathbf{c}^{\text{KV}})^\top$  的全局值可以通过局部计算来近似。我们首先证明应用任何正交变换  $U$  不会改变原始 softmax 输出的等价性。根据公式 5，我们很容易得到：

$$\begin{aligned} Q(\hat{\mathbf{c}}^{\text{KV}})^\top &= QU(\hat{\mathbf{c}}^{\text{KV}}U)^\top = \mathbf{q}(U^\top W^{UK})^\top (\hat{\mathbf{c}}^{\text{KV}}U)^\top \\ &= Q'(\hat{\mathbf{c}}^{\text{KV}}U)^\top \end{aligned} \quad (21)$$

类似于 4.1 节，通过将  $U$  等效地吸收到  $W^{UKV}$  和  $W^{DKV}$  中，吸收到  $Q$  中以获得  $Q'$ ，我们可以施加正交变换  $U$ ，它保留了原始的 softmax 计算并且必须满足：

条件 2（Softmax 切片条件）。

$$Q'(\hat{\mathbf{c}}^{\text{KV}}U)^\top \approx \mu Q'_0(\hat{\mathbf{c}}_0^{\text{KV}}U)^\top \approx \nu Q'_1(\hat{\mathbf{c}}_1^{\text{KV}}U)^\top \quad (22)$$

因此，通过确定系数  $\mu$  和  $\nu$ ，每个设备可以执行其本地计算并按该因子进行缩放，从而逼近全局值。

### 4.3 重新参数化方法

根据上面的推导，我们需要找到一个应用于投影权重的正交变换矩阵  $U$ ，确保变换满足条件 1 和条件 2——局部计算可以准确地逼近全局 RMSNorm 和 softmax 值。为了实现这一目标，我们探索了两种潜在的方法：哈达玛矩阵变换和主成分分析（PCA）。

#### 4.3.1 阿达玛矩阵变换

Hadamard 矩阵是一个特殊的正交矩阵，其中每个条目不是 +1 就是 -1。它通过平衡数字来运行，从而减少极端的数值偏差并促进数据分布更加均匀。在实践中，我们通常使用函数 `scipy.linalg.hadamard(d)` 生成 Sylvester 型 Hadamard 矩阵（也称为 Walsh-Hadamard 矩阵） $H_d \in \mathbb{R}^{d \times d}$ ，使用确定性递归规则构造：

$$H_{2n} = \begin{pmatrix} H_n & H_n \\ H_n & -H_n \end{pmatrix}, \quad H_1 = (1). \quad (23)$$

为了提高鲁棒性，随机对角矩阵  $D$ （其条目取自  $\pm 1$ ）与  $H_d$  相乘，从而在保持正交性的同时打破了确定性结构。自  $H_d H_d^\top = d \cdot \mathbf{I}$  起，正交性是通过按  $\frac{1}{\sqrt{d}}$  缩放  $H_d$  来实现的，确保保留归一化值。

举一个说明性的例子。考虑 4 维向量  $\mathbf{c} = (100, 0, 0, 0)$  和  $4 \times 4$  Hadamard 矩阵  $H_4$ 。变换后的向量  $\mathbf{c}' = \mathbf{c}H_4$  为：

$$\mathbf{c}' = (100, 0, 0, 0) \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} = (50, 50, 50, 50). \quad (24)$$

When applied to an input vector  $\mathbf{c}$ :  $\mathbf{c}' = \mathbf{c} H_d$ ,  $d = 4d_h$ , we obtain:  $\frac{\|(\mathbf{c}H_d)_1\|_2^2}{d/2} \approx \frac{\|(\mathbf{c}H_d)_2\|_2^2}{d/2} \approx \frac{\|\mathbf{c}H_d\|_2^2}{d} = \frac{\|\mathbf{c}\|_2^2}{d}$ , satisfies our key Condition 1 and easily determine  $\alpha = 2$ . This uniformity minimizes approximation error in tensor-parallel RMSNorm, validated experimentally in Figure 2.

However, satisfying Condition 2 is more challenging. While the magnitudes of the Hadamard transformed vector elements are balanced, due to the presence of both positive and negative signs, this transformation does not guarantee that the multiplication of the two parts will be approximately clear. To illustrate this, consider the following example: let  $Q = (100, 0, 0, 0)$  and  $\mathbf{c} = (0, 0, 80, 0)$ . After applying the Hadamard transformation, we have:

$$\begin{aligned} Q' &= QH_4 = (50, 50, 50, 50), \\ \mathbf{c}' &= \mathbf{c}H_4 = (40, 40, -40, -40). \end{aligned}$$

The element-wise product is:  $Q \cdot \mathbf{c}' = (200, 200, -200, -200)$ . When this product is split into two parts, we get:  $400 \neq -400 \neq 0$ . This demonstrates that a standard Hadamard transformation cannot ensure Condition 2. A potential direction to address this issue is to search for an optimized Hadamard matrix via dimension permutations that minimizes the discrepancy between partitions. We leave the investigation of such optimized transformations for future work.

### 4.3.2 Principal Component Analysis (PCA)

PCA is a widely used technique in statistics and machine learning for dimensionality reduction, feature extraction, etc. It transforms a dataset into a new coordinate system such that the greatest variances of the data are captured along the new axes (principal components). Each subsequent component is orthogonal to (i.e., uncorrelated with) the preceding ones. In our context, we leverage this property to project data onto orthogonal dimensions, with the eigenvalues indicating the variance captured along each eigenvector and thus reflecting the statistical importance of each dimension. Moreover, for mean-centered features, the variance is equivalent to mean of the squared values, closely related to squared RMS value.

To implement this, we process a calibration dataset (e.g., Wikitext-2) to collect the KV latent cache (excluding position features) represented by  $F \in \mathbb{R}^{(B \cdot L) \times d}$ . We then compute the eigenvectors  $U$  and eigenvalues  $\Lambda$  by performing eigenvalue decomposition on the covariance matrix  $\Sigma_F = U\Lambda U^\top$ .

Based on Condition 1, we define  $\alpha$  as proportion of variance captured by the first  $d/2$  principal components. Similarly,  $\beta$  represents the proportion of variance captured by the remaining components. These ratios are as follows:

$$\alpha = \frac{\sum_{i=1}^{d/2} \lambda_i}{\sum_{i=1}^d \lambda_i}, \quad \beta = \frac{\sum_{i=d/2+1}^d \lambda_i}{\sum_{i=1}^d \lambda_i}. \quad (25)$$

For Condition 2, the metrics  $\mu$  and  $\nu$  are defined in the same manner, making them equivalent to  $\alpha$  and  $\beta$ , respectively.

## 4.4 TPLA as a Special Case of GLA

Tensor parallelism in GLA employs a two-dimensional sharding scheme, splitting both head axis  $h_q$  and the latent dimension axis  $4d_h$  across devices. For a query tensor  $Q \in \mathbb{R}^{B \times L \times h_q \times 4d_h}$ , this partitioning yields four logical sub-tensors:

$$Q = \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix}, \quad \text{where } Q_{i,j} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}.$$

In standard GLA, they are distributed with only two devices. Thus, only two diagonal blocks can be materialized locally—one per device—without additional communication:

$$\begin{cases} \text{Device 0: } Q_{0,0} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}, \\ \text{Device 1: } Q_{1,1} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}. \end{cases}$$

Each latent slice (of width  $2d_h$ ) is paired with only half of the query heads ( $h_q/2$ ) and thus unable to access the off-diagonal head slices  $Q_{1,0}$  and  $Q_{0,1}$ . In effect, these parts do not contribute to the computation, resulting in significant performance degradation.

In contrast, TPLA overcomes this limitation by enabling each partitioned latent vector to attend to all query heads. It achieves this by reformulating the computation to be algebraically equivalent to a GLA system with double the number of heads. Concretely, define a conceptual query tensor  $Q'$  that duplicates the original query along the head dimension:

$$Q' = \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \\ Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix} \in \mathbb{R}^{B \times L \times (2h_q) \times (4d_h)},$$

当应用于输入向量  $\mathbf{c}$ :  $\mathbf{c}' = \mathbf{c} H_d$ ,  $d = 4d_h$  时, 我们得到:  $\frac{\|(\mathbf{c} H_d)_1\|_2^2}{d/2} \approx \frac{\|(\mathbf{c} H_d)_2\|_2^2}{d/2} \approx \frac{\|\mathbf{c} H_d\|_2^2}{d} = \frac{\|\mathbf{c}\|_2^2}{d}$ , 满足我们的关键条件 1 并轻松确定  $\alpha = 2$ 。这种均匀性最大限度地减少了张量并行 RMSNorm 中的近似误差, 在图 2 中通过实验进行了验证。

然而, 满足条件 2 更具挑战性。虽然哈达玛变换后的向量元素的大小是平衡的, 但由于同时存在正号和负号, 该变换并不能保证两部分的乘法大致清晰。为了说明这一点, 请考虑以下示例: 令  $Q = (100, 0, 0, 0)$  和  $\mathbf{c} = (0, 0, 80, 0)$ 。应用哈达玛变换后, 我们有:

$$\begin{aligned} Q' &= Q H_4 = (50, 50, 50, 50), \\ \mathbf{c}' &= \mathbf{c} H_4 = (40, 40, -40, -40). \end{aligned}$$

逐元素乘积为:  $Q \cdot \mathbf{c}' = (200, 200, -200, -200)$ 。当将此乘积分为两部分时, 我们得到:  $400 \neq -400 \neq 0$ 。这表明标准哈达玛变换无法确保条件 2。解决此问题的一个潜在方向是通过最小化维度排列来搜索优化的哈达玛矩阵分区之间的差异。我们将对这种优化转换的研究留到未来的工作中。

#### 4.3.2 主成分分析 (PCA)

PCA 是统计和机器学习中广泛使用的技术, 用于降维、特征提取等。它将数据集转换为新的坐标系, 以便沿着新轴 (主成分) 捕获数据的最大方差。每个后续分量都与前面的分量正交 (即不相关)。在我们的上下文中, 我们利用此属性将数据投影到正交维度上, 特征值表示沿每个特征向量捕获的方差, 从而反映每个维度的统计重要性。此外, 对于以均值为中心的特征, 方差相当于均方值, 与均方根值密切相关。

为了实现这一点, 我们处理一个校准数据集 (例如, Wikitext-2) 来收集由  $F \in \mathbb{R}^{(B \cdot L) \times d}$  表示的 KV 潜在缓存 (不包括位置特征)。然后, 我们通过对协方差矩阵  $\Sigma_F = U \Lambda U^\top$  执行特征值分解来计算特征向量  $U$  和特征值  $\Lambda$ 。

基于条件 1, 我们将  $\alpha$  定义为前  $d/2$  个主成分捕获的方差比例。类似地,  $\beta$  表示其余分量捕获的方差比例。这些比率如下:

$$\alpha = \frac{\sum_{i=1}^{d/2} \lambda_i}{\sum_{i=1}^d \lambda_i}, \quad \beta = \frac{\sum_{i=d/2+1}^d \lambda_i}{\sum_{i=1}^d \lambda_i}. \quad (25)$$

对于条件 2, 度量  $\mu$  和  $\nu$  以相同的方式定义, 使它们分别相当于  $\alpha$  和  $\beta$ 。

#### 4.4 TPLA 作为 GLA 的特例

GLA 中的张量并行性采用二维分片方案, 跨设备分割头轴  $h_q$  和潜在维度轴  $4d_h$ 。对于查询张量  $Q \in \mathbb{R}^{B \times L \times h_q \times 4d_h}$ , 此分区产生四个逻辑子张量:

$$Q = \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix}, \quad \text{where } Q_{i,j} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}.$$

在标准 GLA 中, 它们仅分布在两个设备上。因此, 只有两个对角块可以在本地实现 (每个设备一个), 而无需额外的通信:

$$\begin{cases} \text{Device 0: } Q_{0,0} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}, \\ \text{Device 1: } Q_{1,1} \in \mathbb{R}^{B \times L \times \frac{h_q}{2} \times 2d_h}. \end{cases}$$

每个潜在切片 (宽度为  $2d_h$ ) 仅与一半的查询头 ( $h_q/2$ ) 配对, 因此无法访问非对角线头切片  $Q_{1,0}$  和  $Q_{0,1}$ 。实际上, 这些部分对计算没有贡献, 导致性能显著下降。

相比之下, TPLA 通过使每个分区的潜在向量能够处理所有查询头来克服这一限制。它通过将计算重新表述为在代数上等同于具有双倍头数的 GLA 系统来实现这一点。具体来说, 定义一个概念查询张量  $Q'$ , 它沿头维度复制原始查询:

$$Q' = \begin{pmatrix} Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \\ Q_{0,0} & Q_{0,1} \\ Q_{1,0} & Q_{1,1} \end{pmatrix} \in \mathbb{R}^{B \times L \times (2h_q) \times (4d_h)},$$

For  $h_q$  original heads, TPLA’s duplication additionally creates  $h_q$  heads. This is algebraically equivalent to a GLA system with  $2h_q$  heads  $4d_h$  latent dimension. Thus, we can perfectly follow the same as TPLA sharding way. When split into two device, we have:

$$\begin{cases} \text{Device 0: } [Q_{0,0} & Q_{0,1}] \in \mathbb{R}^{B \times L \times h_q \times 4d_h} \\ \text{Device 1: } [Q_{1,0} & Q_{1,1}] \in \mathbb{R}^{B \times L \times h_q \times 4d_h} \end{cases}$$

Generalizing to  $k$  devices, let  $g$  denote the TPLA replication factor (number of latent-cache-slice groups). TPLA divides  $k$  devices into  $r$  group size of size  $k/g$ . Each group holds a disjoint slice of the latent axis of width  $4d_h/g$  and replicates the complete set of head parameters. Within each group, the head axis is sharded across the  $k/g$  devices. Consequently, each device processes  $\frac{h_q}{k/g}$  heads, and  $\frac{4d_h}{g}$  latent features. For  $g = 2$ ,  $k = 2$ , this recovers the two-device case above, where each device receives  $h_q$  heads and  $2d_h$  latent width. The computational complexity arising from parameter replication is analyzed in Section 4.5.

In summary, because TPLA preserves GLA’s sharding pattern—differing only by a constant-factor replication of head parameters—state-of-the-art attention optimizations (e.g., FlashAttention-3) can be applied to TPLA without substantial changes to the underlying framework.

#### 4.5 Prefill-Decode Separation

Large language model inference is usually into two phases with distinct performance characteristics: *prefill* and *decode*. The prefill phase processes the entire prompt in a single, parallel pass to compute the initial Key-Value (KV) cache. This large-batch computation is fundamentally compute-bound. The subsequent decode phase autoregressively generates one token at a time. Each generation step requires reading the entire, growing KV cache from high-bandwidth memory (HBM) to on-chip SRAM. As the context length increases, this large data transfer becomes the primary bottleneck, making the decode phase memory-bound.

Our proposed technique, TPLA, addresses this challenge by reducing the KV cache size on each device. This reduction effectively alleviates the memory bandwidth bottleneck at the cost of a minor increase in computation. In essence, TPLA shifts the decode phase from being memory-bound towards being more compute-bound. A detailed analysis is as below.

**Complexity Analysis of TPLA** To maximize the degree of tensor parallelism and enable acceleration methods compatible with GLA, TPLA requires the replication of head-specific parameters across latent attention groups, as discussed in Section 4.4. Specifically, let’s analyze the case with a tensor parallelism (TP) degree of 2. We consider a hidden state  $X \in \mathbb{R}^{L_q \times D}$  for a single-batch inference using the MLA-absorbing strategy. The dominant cost lies in the attention computation. For a KV cache of length  $S_{kv}$ , the complexity of the TPLA attention module (Equation 6) is approximately  $\mathcal{O}(L_q \times S_{KV} \times h_q \times 2d_h \times 2)$ . In comparison, for MLA (Equation 1), with TP=2, the heads are split into two groups of  $\frac{h_q}{2}$ , leading to a complexity of  $\mathcal{O}(L_q \times S_{KV} \times \frac{h_q}{2} \times 4d_h \times 2)$ . These two complexities are arithmetically equivalent. (Strictly speaking, positional components introduce additional overhead, since TPLA doubles the number of heads without reducing the RoPE dimension, but this effect is relatively minor.) Similarly, the  $\tilde{O}$  computations are also equivalent.

Beyond the main attention computation, TPLA modifies other calculations: the computation of  $c^{KV}$  in TPLA is distributed across two devices, reducing complexity by  $\mathcal{O}(L_q \times D \times 2d_h \times 2)$ , while the computation of  $Q$  increases by  $\mathcal{O}(L_q \times D \times 2h_q \times d_h)$ . However, as context length grows, the overall cost is increasingly dominated by the self-attention module (see Figure 4).

To mitigate the additional computational overhead of TPLA, we also strategically decouple the attention mechanisms: retaining standard MLA during compute-intensive prefilling to minimize computation and reduce loss caused by converting MLA to TPLA, while activating TPLA exclusively during memory-bound decoding to minimize KV cache footprint. This hybrid approach thereby further optimizes performance by matching each phase to its most suitable mechanism.

**Discussion** The above analysis focuses on MLA-absorbing computation. However, when constructing the full-size KV cache and performing attention, TPLA doubles the number of heads while maintaining the same head dimension  $d_h$ . As a result, the overall computational load increases, making training TPLA from scratch potentially costly. Designing effective and efficient training strategies for TPLA remains an open problem, which we leave for future work. Nevertheless, one practical pathway is to adopt the existing MLA design during training and then convert it to TPLA with only minimal loss. This approach allows us to retain training efficiency while still benefiting from TPLA’s advantages in inference scenarios.

对于 $h_q$ 原始头，TPLA的复制额外创建 $h_q$ 头。这在代数上等价于具有 $2h_q$ 个头和 $4d_h$ 潜在维度的GLA系统。这样，我们就可以完全遵循TPLA的分片方式了。当分成两个设备时，我们有：

$$\begin{cases} \text{Device 0: } [Q_{0,0} & Q_{0,1}] \in \mathbb{R}^{B \times L \times h_q \times 4d_h} \\ \text{Device 1: } [Q_{1,0} & Q_{1,1}] \in \mathbb{R}^{B \times L \times h_q \times 4d_h} \end{cases}$$

推广到 $k$ 设备，让 $g$ 表示TPLA复制因子（潜在缓存片组的数量）。TPLA将 $k$ 设备划分为大小为 $k/g$ 的 $r$ 组。每组持有宽度为 $4d_h/g$ 的潜轴的不相交切片，并复制完整的头部参数集。在每个组中，头部轴在 $k/g$ 设备上分片，因此，每个设备处理 $\frac{h_q}{k/g}$ 头部和 $\frac{4d_h}{g}$ 潜在特征。对于 $g=2, k=2$ ，这恢复了上面的两个设备的情况，其中每个设备接收 $h_q$ 个头和 $2d_h$ 潜在宽度。参数复制产生的计算复杂度在4.5节中进行了分析。

总之，由于TPLA保留了GLA的分片模式（仅通过头参数的恒定因子复制而有所不同），因此可以将最先进的注意力优化（例如FlashAttention-3）应用于TPLA，而无需对底层框架进行实质性更改。

#### 4.5 预填充-解码分离

大型语言模型推理通常分为两个具有不同性能特征的阶段：*prefill*和*decode*。预填充阶段在单个并行传递中处理整个提示，以计算初始键值(KV)缓存。这种大批量计算从根本上说是受计算限制的。随后的解码阶段每次自回归生成一个令牌。每个生成步骤都需要读取整个不断增长的KV缓存，从高带宽内存(HBM)到片上SRAM。随着上下文长度的增加，这种大数据传输成为主要瓶颈，使解码阶段受到内存限制。

我们提出的技术TPLA通过减少每个设备上的KV缓存大小来解决这一挑战。这种减少有效缓解了内存带宽瓶颈，但代价是计算量略有增加。从本质上讲，TPLA将解码阶段从受内存限制转变为更受计算限制。详细分析如下。

**TPLA的复杂性分析** 为了最大化张量并行度并启用与GLA兼容的加速方法，TPLA需要跨潜在注意力组复制头部特定参数，如4.4节中所述。具体来说，让我们分析张量并行度(TP)为2的情况。我们考虑使用MLA吸收策略进行单批次推理的隐藏状态 $X \in \mathbb{R}^{L_q \times D}$ 。主要成本在于注意力计算。对于长度为 $S_{kv}$ 的KV缓存，TPLA注意力模块（公式6）的复杂度约为 $\mathcal{O}(L_q \times S_{kv} \times h_q \times 2d_h \times 2)$ 。相比之下，对于MLA（等式1），使用TP=2，头被分成两组 $\frac{h_q}{2}$ ，导致复杂度为 $\mathcal{O}(L_q \times S_{kv} \times \frac{h_q}{2} \times 4d_h \times 2)$ 。这两种复杂性在算术上是等价的。（严格来说，位置组件引入了额外的开销，因为TPLA在不减少RoPE维度的情况下使头数量加倍，但这种影响相对较小。）类似地， $\tilde{O}$ 计算也是等效的。

除了主要的注意力计算之外，TPLA还修改了其他计算：TPLA中 $c^{kv}$ 的计算分布在两个设备上，将复杂度降低了 $\mathcal{O}(L_q \times D \times 2d_h \times 2)$ ，而 $Q$ 的计算则增加了 $\mathcal{O}(L_q \times D \times 2h_q \times d_h)$ 。然而，随着上下文长度的增长，整体成本越来越多地由自注意力模块主导（见图4）。

为了减轻TPLA的额外计算开销，我们还策略性地解耦注意力机制：在计算密集型预填充期间保留标准MLA，以最大限度地减少计算量并减少将MLA转换为TPLA造成的损失，同时在内存绑定解码期间专门激活TPLA，以最大限度地减少KV缓存占用空间。因此，这种混合方法通过将每个阶段与其最合适的机制相匹配来进一步优化性能。

讨论上述分析重点关注MLA吸收计算。然而，在构建全尺寸KV缓存并执行注意力时，TPLA将头数量加倍，同时保持相同的头尺寸 $d_h$ 。因此，整体计算负载增加，使得从头开始训练TPLA的成本可能很高。为TPLA设计有效且高效的培训策略仍然是一个悬而未决的问题，我们将其留给未来的工作。尽管如此，一种实用的途径是在训练期间采用现有的MLA设计，然后将其转换为TPLA，损失最小。这种方法使我们能够保持训练效率，同时仍然受益于TPLA在推理场景中的优势。

Table 1: WikiText-2 Perplexity and Commonsense reasoning performance when converting the MLA to TPLA. The six benchmarks include MMLU, ARC (easy and challenge), PIQA, HellaSwag, OpenBookQA (OBQA), and Winogrande(WG).

| Model            | PPL↓  | Avg.↑ | MMLU  | ARC   | PIQA  | HellaSwag | WG    | OBQA  |
|------------------|-------|-------|-------|-------|-------|-----------|-------|-------|
| DeepSeek-V2-Lite | 6.31  | 61.75 | 43.19 | 60.39 | 80.20 | 74.46     | 65.43 | 45.80 |
| - GLA            | 2212. | 33.77 | 25.32 | 26.77 | 51.47 | 25.65     | 49.88 | 23.60 |
| - TPLA           | 7.24  | 54.33 | 37.67 | 51.50 | 75.46 | 63.56     | 59.19 | 38.60 |
| - TPLA (align)   | 6.51  | 61.52 | 42.72 | 62.58 | 79.82 | 73.32     | 65.90 | 44.80 |
| - TPLA (pd sep.) | 6.31  | 61.44 | 43.19 | 60.14 | 80.09 | 74.41     | 65.59 | 45.20 |
| DeepSeek-V2      | 3.89  | 68.32 | 51.91 | 69.09 | 83.13 | 82.17     | 74.03 | 49.60 |
| - TPLA           | 4.72  | 63.40 | 47.19 | 65.04 | 80.69 | 75.46     | 66.61 | 45.40 |
| DeepSeek-V3      | 3.24  | 72.10 | 60.85 | 77.16 | 85.58 | 85.41     | 75.22 | 48.40 |
| - TPLA           | 4.02  | 68.00 | 54.88 | 75.25 | 82.70 | 80.69     | 69.46 | 45.00 |
| Kimi-K2-Base     | 1.91  | 73.52 | 63.20 | 78.75 | 85.47 | 87.55     | 75.93 | 50.20 |
| - TPLA           | 2.44  | 70.49 | 57.64 | 76.00 | 83.79 | 83.53     | 72.38 | 49.60 |
| LLaMA-2-7B       | 5.47  | 59.85 | 41.43 | 59.24 | 78.40 | 73.29     | 64.96 | 41.80 |
| - TransMLA       | 5.88  | 58.95 | 40.38 | 57.64 | 78.18 | 70.59     | 62.90 | 44.00 |
| - TPLA           | 6.74  | 54.68 | 36.12 | 53.21 | 74.81 | 64.52     | 59.04 | 40.40 |

## 5 Experiment

An advantage of TPLA over GLA [20] is that TPLA can be applied without training a model from scratch. It allows direct loading of models originally trained with MLA (e.g., the DeepSeek series [53, 17, 19], Kimi-k2 [54], TransMLA [16]), and—through our proposed reparameterization method and Prefill/Decode Separation technology—mitigates performance degradation caused by changes in the attention mechanism.

### 5.1 Performance on Commonsense Tasks

In this section, we evaluate TPLA by directly loading MLA checkpoints *without any additional training* on short-text commonsense tasks. Performance is measured with the LightEval framework on MMLU [55], ARC (Easy/Challenge) [56], PIQA [57], HellaSwag [58], OpenBookQA (OBQA) [59], and WinoGrande (WG) [60]. Results are reported in Table 1. For **GLA**, following the procedure in Section 3.2, we partition the attention heads into two groups, assigning each group half of the latent dimension. As shown in Table 1, discarding half of each head’s KV cache causes severe performance degradation—for example, WikiText-2 perplexity (ppl) increases from 6.31 with MLA to 2212 with GLA—whereas **TPLA**, which allows each attention head to use the full latent dimension across different devices, maintains a ppl of 7.24. This comparison indicates that TPLA preserves MLA’s representational capacity while reducing the per-device KV-cache footprint. We therefore expect that pretraining TPLA from scratch would outperform GLA. For **TPLA**, we first use WikiText-2 [61] as a calibration set and, following Sections 4.1 and 4.2, slice the MLA components (the  $KV_a$  RMSNorm and the softmax) to obtain TPLA weights. As shown in Table 1, this requires no fine-tuning and yields only minor accuracy degradation. The reparameterization method used here is the PCA-based approach described in Section 4.3.2. For **TPLA (align)**, we use the SmolLM-Corpus [62] for lightweight alignment. First, we match the layer-wise input/output features of TPLA to those of the original MLA model using 256 random samples of length 2,048 for 10 epochs, minimizing MSE with the Muon optimizer (initial learning rate  $1e-6$ ). Next, we align the end-to-end model outputs using 100M tokens, following the TransMLA setting (batch size = 32, learning rate =  $2e-5$ , warmup ratio = 0.03, cosine scheduler, max sequence length = 4096). Experiments are conducted on a node with  $8 \times$  GPUs (96 GB per GPU,  $\sim 148$  FP16 TFLOPS each). This small amount of alignment data is sufficient to recover the performance of the converted model. For **TPLA (PD-sep.)**, we use MLA in the prefilling stage with the same reparameterization but *without* slicing the RMSNorm or softmax; prefilling thus behaves identically to the original MLA, and the KV cache can be partially reused by TPLA during decoding. By avoiding slicing for most tokens, this prefill–decode separation achieves performance close to the original model *without any training*. For **LLaMA-2-7B**, we first apply TransMLA [16] to convert MHA/GQA to MLA (64 RoPE dimensions and 512 NoPE dimensions—corresponding to a pruning ratio of 92.97%.) and then fine-tune to recover performance. We subsequently convert the MLA checkpoint released by TransMLA directly into TPLA. With TransMLA as a bridge, TPLA can be applied to pretrained models that originally use MLA, GQA, or MHA.

表 1: 将 MLA 转换为 TPLA 时的 WikiText-2 困惑度和常识推理性能。六个基准测试包括 MMLU、ARC (简单和挑战)、PIQA、HellaSwag、OpenBookQA (OBQA) 和 Winogrande(WG)。

| Model            | PPL↓  | Avg.↑ | MMLU  | ARC   | PIQA  | HellaSwag | WG    | OBQA  |
|------------------|-------|-------|-------|-------|-------|-----------|-------|-------|
| DeepSeek-V2-Lite | 6.31  | 61.75 | 43.19 | 60.39 | 80.20 | 74.46     | 65.43 | 45.80 |
| - GLA            | 2212. | 33.77 | 25.32 | 26.77 | 51.47 | 25.65     | 49.88 | 23.60 |
| - TPLA           | 7.24  | 54.33 | 37.67 | 51.50 | 75.46 | 63.56     | 59.19 | 38.60 |
| - TPLA (align)   | 6.51  | 61.52 | 42.72 | 62.58 | 79.82 | 73.32     | 65.90 | 44.80 |
| - TPLA (pd sep.) | 6.31  | 61.44 | 43.19 | 60.14 | 80.09 | 74.41     | 65.59 | 45.20 |
| DeepSeek-V2      | 3.89  | 68.32 | 51.91 | 69.09 | 83.13 | 82.17     | 74.03 | 49.60 |
| - TPLA           | 4.72  | 63.40 | 47.19 | 65.04 | 80.69 | 75.46     | 66.61 | 45.40 |
| DeepSeek-V3      | 3.24  | 72.10 | 60.85 | 77.16 | 85.58 | 85.41     | 75.22 | 48.40 |
| - TPLA           | 4.02  | 68.00 | 54.88 | 75.25 | 82.70 | 80.69     | 69.46 | 45.00 |
| Kimi-K2-Base     | 1.91  | 73.52 | 63.20 | 78.75 | 85.47 | 87.55     | 75.93 | 50.20 |
| - TPLA           | 2.44  | 70.49 | 57.64 | 76.00 | 83.79 | 83.53     | 72.38 | 49.60 |
| LLaMA-2-7B       | 5.47  | 59.85 | 41.43 | 59.24 | 78.40 | 73.29     | 64.96 | 41.80 |
| - TransMLA       | 5.88  | 58.95 | 40.38 | 57.64 | 78.18 | 70.59     | 62.90 | 44.00 |
| - TPLA           | 6.74  | 54.68 | 36.12 | 53.21 | 74.81 | 64.52     | 59.04 | 40.40 |

## 5 实验

TPLA 相对于 GLA [20] 的优点是无需从头开始训练模型即可应用 TPLA。它允许直接加载最初使用 MLA 训练的模型（例如 DeepSeek 系列 [53,17,19]、Kimi-k2 [54]、TransMLA [16]），并且通过我们提出的重新参数化方法和预填充/解码分离技术，减轻由于注意力机制变化而导致的性能下降。

### 5.1 常识性任务的表现

在本节中，我们通过在短文本常识任务上直接加载 MLA 检查点 *without any additional training* 来评估 TPLA。性能是通过 MMLU [55]、ARC (Easy/Challenge) [56]、PIQA [57]、HellaSwag [58]、OpenBookQA (OBQA) [59] 和 WinoGrande (WG) [60] 上的 LightEval 框架来测量的。结果如表 1 所示。对于 GLA，按照第 3.2 节中的过程，我们将注意力头分为两组，为每组分配一半的潜在维度。如表 1 所示，丢弃每个头的 KV 缓存的一半会导致严重的性能下降，例如，WikiText-2 困惑度 (ppl) 从 MLA 的 6.31 增加到 GLA 的 2212，而 TPLA 允许每个注意力头跨不同设备使用完整的潜在维度，将 ppl 维持在 7.24。此比较表明 TPLA 保留了 MLA 的表示能力，同时减少了每个设备的 KV 缓存占用空间。因此，我们预计从头开始预训练 TPLA 将优于 GLA。对于 TPLA，我们首先使用 WikiText-2 [61] 作为校准集，然后按照第 4.1 节和第 4.2 节，对 MLA 分量 ( $KV_a$  RMSNorm 和 softmax) 进行切片以获得 TPLA 权重。如表 1 所示，这不需要微调，并且只会导致较小的精度下降。这里使用的重新参数化方法是第 4.3.2 节中描述的基于 PCA 的方法。对于 TPLA (对齐)，我们使用 SmolLM-Corpus [62] 进行轻量级对齐。首先，我们使用长度为 2,048 的 256 个随机样本进行 10 个周期，将 TPLA 的逐层输入/输出特征与原始 MLA 模型的输入/输出特征进行匹配，并使用 Muon 优化器最小化 MSE（初始学习率  $1e-6$ ）。接下来，我们使用 100M 令牌对齐端到端模型输出，遵循 TransMLA 设置（批量大小 = 32、学习率 =  $2e-5$ 、预热比率 = 0.03、余弦调度程序、最大序列长度 = 4096）。实验在具有 8 个  $\times$  GPU 的节点上进行（每个 GPU 96 GB，每个  $\sim 148$  FP16 TFLOPS）。这一少量的对齐数据足以恢复转换后模型的性能。对于 TPLA (PD-sep.)，我们在预填充阶段使用 MLA，具有相同的重新参数化，但 *without* 对 RMSNorm 或 softmax 进行切片；因此，预填充的行为与原始 MLA 相同，并且 KV 缓存可以在解码期间被 TPLA 部分重用。通过避免对大多数令牌进行切片，这种预填充-解码分离实现了接近原始模型 *without any training* 的性能。对于 LLaMA-2-7B，我们首先应用 TransMLA [16] 将 MHA/GQA 转换为 MLA（64 个 RoPE 维度和 512 个 NoPE 维度——对应于 92.97% 的剪枝率），然后进行微调以恢复性能。随后我们将 TransMLA 发布的 MLA 检查点直接转换为 TPLA。以 TransMLA 为桥梁，TPLA 可以应用于原本使用 MLA、GQA 或 MHA 的预训练模型。

Table 2: Longbench performance when converting the MLA to TPLA.

| Model                   | Avg.  | MultiQA | SingleQA | Summarize | Few-Shot | Code  | Synthetic |
|-------------------------|-------|---------|----------|-----------|----------|-------|-----------|
| DeepSeek-V2-Lite        | 28.90 | 12.43   | 20.04    | 16.74     | 62.59    | 57.86 | 3.77      |
| - <i>TPLA</i>           | 10.98 | 6.96    | 9.20     | 6.91      | 25.29    | 14.41 | 3.11      |
| - <i>TPLA (align)</i>   | 22.60 | 10.97   | 14.67    | 16.59     | 58.03    | 31.59 | 3.77      |
| - <i>TPLA (pd sep.)</i> | 24.44 | 13.95   | 15.07    | 8.62      | 59.10    | 46.67 | 3.23      |
| DeepSeek-V3             | 58.19 | 55.37   | 51.65    | 23.97     | 69.42    | 80.09 | 68.63     |
| - <i>TPLA</i>           | 44.52 | 35.02   | 38.53    | 12.55     | 53.01    | 61.20 | 66.83     |
| - <i>TPLA (pd sep.)</i> | 56.04 | 53.01   | 50.17    | 21.39     | 67.22    | 75.97 | 68.47     |

These experiments demonstrate that converting MLA-based models to TPLA can effectively preserve performance. Given the benefits of TPLA for tensor parallelism, this presents a promising approach for efficient model deployment and acceleration.

## 5.2 Performance on Longbench Tasks

As context length grows, memory traffic increases and the KV-cache size becomes a primary driver of latency and throughput. To assess how TPLA converted from MLA behaves on long inputs, we evaluate on *LongBench* [63], a bilingual (English/Chinese), multi-task benchmark for long-context understanding that comprises 21 tasks across six categories (e.g., question answering, summarization, and few-shot learning). Because long-text inference is slower, we report results only for DeepSeek-V2-Lite and DeepSeek-V3. Due to GPU memory constraints, the maximum input context length is set to 31,500 tokens for DeepSeek-V2-Lite and 127,500 tokens for DeepSeek-V3. For each task, the output length is kept the same as in the original paper. The outcomes are summarized in Table 2. We observe that slicing errors in RMSNorm and softmax accumulate with sequence length, leading to some degradation on LongBench. **TPLA (align)** follows the same alignment recipe as in the previous section, but its effectiveness is limited because the alignment corpus is formed by concatenating short texts. In contrast, **TPLA (PD-sep.)** adopts a prefill–decode separation: MLA is used unchanged in the prefill stage (no slicing of RMSNorm/softmax), and the resulting KV cache is partially reused by TPLA during decoding, which reduces first-token latency and accuracy loss. On DeepSeek-V2-Lite, the training-free **TPLA (PD-sep.)** surpasses the aligned variant, and on DeepSeek-V3 the model retains strong long-form reasoning with only a modest average drop of 2.15%. These small losses, compared with training from scratch, are likely recoverable with a small amount of additional training.

## 5.3 Ablation Study

### 5.3.1 Part 1

We highlight two structural differences. (i) *Per-head latent capacity*: GLA gives each attention head only half of the latent dimension, whereas TPLA preserves the full latent dimension per head. (ii) *Prefill–decode (PD) separation*: during the compute-intensive prefill stage we keep the reparameterized MLA form *without* splitting RMSNorm or softmax; during decoding, TPLA uses PD separation while consuming the prefill KV cache. We analyze the results in Table 1 to quantify these effects:

**1) MLA → GLA conversion.** Directly converting MLA to GLA forces each attention head to access only half of its original latent representation, causing substantial information loss and a marked accuracy drop across all benchmarks.

**2) Prefill–decode separation.** Avoiding RMSNorm/softmax partitioning in prefill reduces approximation error for the vast majority of tokens. Moreover, the MLA reparameterization enables the prefill KV cache to be used directly by TPLA at decode time, improving both quality and efficiency.

### 5.3.2 Part 2

In Section 4, we identified **RMSNorm** and **softmax** as the primary sources of error when converting MLA to TPLA. To mitigate this, we proposed two reparameterization strategies, **Hadamard-based** and **PCA-based**, to reduce the performance degradation introduced by parallelizing these components. This section presents ablation studies analyzing the impact of each reparameterization method on individual modules. Figure 2 presents ablation results analyzing the effectiveness of each method. The key findings are:

表 2: 将 MLA 转换为 TPLA 时的 Longbench 性能。

| Model            | Avg.  | MultiQA | SingleQA | Summarize | Few-Shot | Code  | Synthetic |
|------------------|-------|---------|----------|-----------|----------|-------|-----------|
| DeepSeek-V2-Lite | 28.90 | 12.43   | 20.04    | 16.74     | 62.59    | 57.86 | 3.77      |
| - TPLA           | 10.98 | 6.96    | 9.20     | 6.91      | 25.29    | 14.41 | 3.11      |
| - TPLA (align)   | 22.60 | 10.97   | 14.67    | 16.59     | 58.03    | 31.59 | 3.77      |
| - TPLA (pd sep.) | 24.44 | 13.95   | 15.07    | 8.62      | 59.10    | 46.67 | 3.23      |
| DeepSeek-V3      | 58.19 | 55.37   | 51.65    | 23.97     | 69.42    | 80.09 | 68.63     |
| - TPLA           | 44.52 | 35.02   | 38.53    | 12.55     | 53.01    | 61.20 | 66.83     |
| - TPLA (pd sep.) | 56.04 | 53.01   | 50.17    | 21.39     | 67.22    | 75.97 | 68.47     |

这些实验表明，将基于 MLA 的模型转换为 TPLA 可以有效地保持性能。鉴于 TPLA 在张量并行性方面的优势，这为高效模型部署和加速提供了一种有前景的方法。

## 5.2 长凳任务的表现

随着上下文长度的增加，内存流量增加，KV 缓存大小成为延迟和吞吐量的主要驱动因素。为了评估从 MLA 转换而来的 TPLA 在长输入上的表现，我们对 *LongBench* [63] 进行了评估，这是一种用于长上下文理解的双语（英语/中文）多任务基准，包括跨 6 个类别的 21 个任务（例如，问答、总结和小样本学习）。由于长文本推理速度较慢，因此我们仅报告 DeepSeek-V2-Lite 和 DeepSeek-V3 的结果。由于 GPU 内存限制，DeepSeek-V2-Lite 的最大输入上下文长度设置为 31,500 个令牌，DeepSeek-V3 的最大输入上下文长度设置为 127,500 个令牌。对于每个任务，输出长度保持与原始论文中的相同。结果总结在表 2 中。我们观察到 RMSNorm 和 softmax 中的切片误差随着序列长度的增加而累积，导致 LongBench 上的一些退化。TPLA (align) 遵循与上一节相同的对齐方法，但其有效性有限，因为对齐语料库是通过连接短文本形成的。相比之下，TPLA (PD-sep.) 采用预填充-解码分离：MLA 在预填充阶段不变地使用（没有 RMSNorm/softmax 切片），并且生成的 KV 缓存在解码期间被 TPLA 部分重用，这减少了第一个令牌延迟和准确性损失。在 DeepSeek-V2-Lite 上，免训练 TPLA (PD-sep.) 超越了对齐变体，而在 DeepSeek-V3 上，该模型保留了强大的长式推理，平均下降幅度仅为 2.15%。与从头开始训练相比，这些微小的损失很可能通过少量的额外训练来恢复。

## 5.3 消融研究

### 5.3.1 第 1 部分

我们强调两个结构性差异。(i) *Per-head latent capacity*: GLA 仅给予每个注意力头一半的潜在维度，而 TPLA 保留每个注意力头的完整潜在维度。(ii) *Prefill-decode (PD) separation*: 在计算密集型预填充阶段，我们保留重新参数化的 MLA 形式 *without* 分割 RMSNorm 或 softmax；在解码过程中，TPLA 使用 PD 分离，同时消耗预填充 KV 缓存。我们分析表 1 中的结果来量化这些影响：

- 1) MLA  $\rightarrow$  GLA 转换。直接将 MLA 转换为 GLA 会迫使每个注意力头仅访问其原始潜在表示的一半，从而导致大量信息丢失，并且所有基准的准确性显著下降。
- 2) 预填充-解码分离。在预填充中避免 RMSNorm/softmax 分区可以减少绝大多数令牌的近似误差。此外，MLA 重新参数化使得 TPLA 在解码时可以直接使用预填充 KV 缓存，从而提高质量和效率。

### 5.3.2 第 2 部分

在第 4 节中，我们将 RMSNorm 和 softmax 确定为将 MLA 转换为 TPLA 时的主要误差源。为了缓解这个问题，我们提出了两种重新参数化策略，即基于 Hadamard 和基于 PCA 的策略，以减少这些组件并行化带来的性能下降。本节介绍了消融研究，分析了每种重新参数化方法对各个模块的影响。图 2 显示了分析每种方法有效性的消融结果。主要发现是：

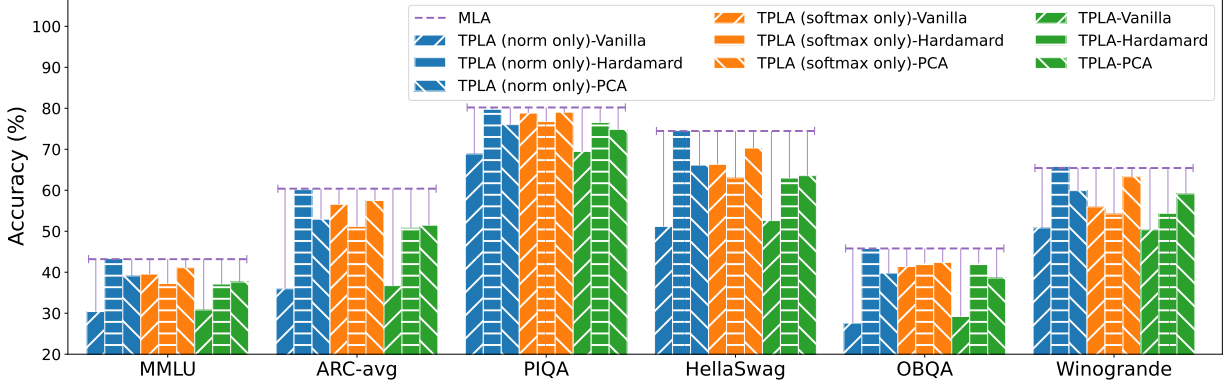


Figure 2: Accuracy across multiple benchmarks under different tensor-parallelism methods (indicated by colors) and reparameterization strategies (indicated by textures). The purple horizontal line marks the original DeepSeek-V2-Lite accuracy, and the vertical bars show each method’s accuracy drop relative to this MLA baseline. **TPLA (norm only)** parallelizes RMSNorm across two devices, followed by an `allgather` before the softmax. **TPLA (softmax only)** applies RMSNorm normally and parallelizes the softmax. **TPLA** parallelizes both RMSNorm and softmax. **Original** splits parameters evenly; **Hadamard** balances parts prior to splitting; **PCA** concentrates information into earlier dimensions before splitting. To better visualize method-induced loss, TPLA results are reported without PD separation in this figure.

**1) Error ordering.** Empirically, slicing *RMSNorm* incurs the least loss, slicing *softmax* is worse, and slicing both is worst:

**2) TP on RMSNorm only:** The Hadamard-based method balances the norm computation across devices effectively, leading to performance comparable to the original MLA model on multiple tasks.

**3) TP on softmax only:** The PCA-based method concentrates information into the dimensions assigned to device 1, effectively preserving performance. In contrast, the Hadamard-based method fails to improve softmax accuracy. We hypothesize that the exponential nature of softmax makes it more sensitive to imbalance. Although Hadamard-based reparameterization achieves statistical balance across devices, small per-sample perturbations may result in significant asymmetries, adversely affecting final performance.

**4) TP over both RMSNorm and softmax.** When both components are parallelized, the PCA-based reparameterization consistently achieves the best performance. Consequently, we adopt this configuration for all experiments in the paper unless otherwise stated.

## 5.4 Inference Speedup with TPLA

### 5.4.1 Decoding Throughput

LLM decoding is often *memory-bandwidth bound*. TPLA splits each attention head’s input dimension across two devices, reducing per-device memory traffic and alleviating the bandwidth bottleneck. We evaluate the speedup of TPLA over MLA on two large models, **DeepSeek-V3-0324** (685B parameters) and **Kimi-K2-Base** (1T parameters). Because these models are extremely large and Mixture-of-Experts (MoE) routing can confound attention-speed effects, we remove MoE for timing. Both models are converted to **BF16**. All experiments use **FlashAttention-3** to ensure a fair comparison.

For **TPLA** with  $TP=2$ , the number of attention heads per device stays unchanged, while the latent dimension changes from  $64+512$  to  $(64+256) \times 2$ , so each device holds a 320-dimensional KV cache. For **MLA** with  $TP=2$ , the latent dimension is unchanged and heads are distributed across devices (e.g., DeepSeek-V3: 64 heads  $\times 2$ ; Kimi-K2: 32 heads  $\times 2$ ). For **MLA** with  $TP>2$ , we continue splitting along heads only. For **TPLA** with  $TP>2$ , we further split heads *in addition to* halving the latent dimension; for example, with  $TP=4$  on Kimi-K2-TPLA, we use 32 heads  $\times 2$  per device with a 320-dimensional latent per head. In this setting, the per-device compute halves, while memory traffic matches  $TP=2$ ; decoding remains memory-bound, so the speedup is similar to  $TP=2$ . Consequently, we report measurements on two GPUs.

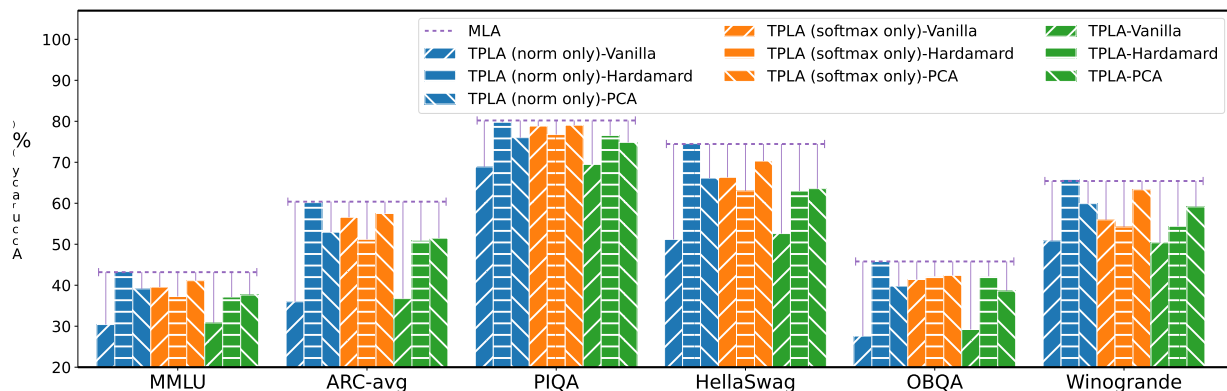


图 2: 不同张量并行方法（由颜色表示）和重新参数化策略（由纹理表示）下多个基准的准确度。紫色水平线标记了原始 DeepSeek-V2-Lite 精度，垂直条显示每种方法相对于该 MLA 基线的精度下降。TPLA（仅范数）在两个设备上并行化 RMSNorm，然后在 softmax 之前进行 allgather。TPLA（仅限 softmax）通常应用 RMSNorm 并并行化 softmax。TPLA 并行化 RMSNorm 和 softmax。原始均分参数；Hadamard 在分割前平衡零件；PCA 在分裂之前将信息集中到较早的维度中。为了更好地可视化方法引起的损失，该图中报告的 TPLA 结果没有 PD 分离。

- 1) 订购错误。根据经验，切片 *RMSNorm* 的损失最小，切片 *softmax* 的损失更差，切片两者的损失最差：
- 2) 仅在 RMSNorm 上进行 TP：基于 Hadamard 的方法有效地平衡了跨设备的范数计算，从而在多个任务上获得与原始 MLA 模型相当的性能。
- 3) 仅在 softmax 上进行 TP：基于 PCA 的方法将信息集中到分配给设备 1 的维度中，有效地保留了性能。相比之下，基于 Hadamard 的方法无法提高 softmax 精度。我们假设 softmax 的指数性质使其对不平衡更加敏感。尽管基于 Hadamard 的重新参数化实现了跨设备的统计平衡，但每个样本的微小扰动可能会导致显著的不对称，从而对最终性能产生不利影响。
- 4) TP 超过 RMSNorm 和 softmax。当两个组件并行化时，基于 PCA 的重新参数化始终能够实现最佳性能。因此，除非另有说明，我们在本文中的所有实验中都采用这种配置。

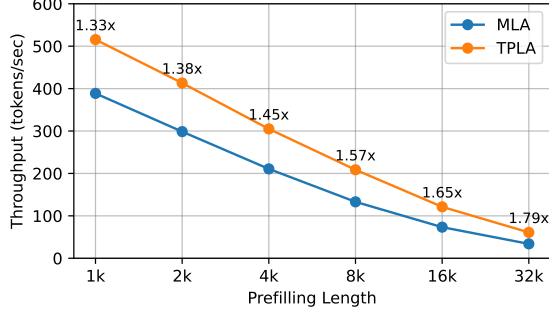
## 5.4 使用 TPLA 进行推理加速

### 5.4.1 解码吞吐量

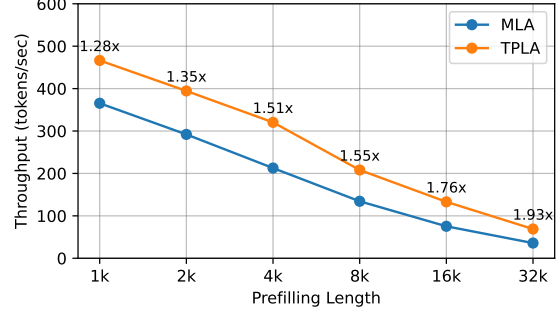
LLM 解码通常是 *memory-bandwidth bound*。TPLA 将每个注意力头的输入维度分割到两个设备上，减少每个设备的内存流量并缓解带宽瓶颈。我们在两个大型模型 DeepSeek-V3-0324（685B 参数）和 Kimi-K2-Base（1T 参数）上评估了 TPLA 相对于 MLA 的加速。由于这些模型非常大，并且专家混合 (MoE) 路由可能会混淆注意力速度效应，因此我们删除了 MoE 以进行计时。两种型号均转换为 BF16。所有实验均使用 FlashAttention-3 以确保公平比较。

对于 TP=2 的 TPLA，每个设备的注意力头数量保持不变，而潜在维度从  $64+512$  变为  $(64+256) \times 2$ ，因此每个设备拥有 320 维的 KV 缓存。对于具有 TP=2 的 MLA，潜在维度不变，并且头分布在设备之间（例如，DeepSeek-V3: 64 个头  $\times 2$ ；Kimi-K2: 32 个头  $\times 2$ ）。对于具有 TP>2 的 MLA，我们继续仅沿着头部进行分裂。对于具有 TP>2 的 TPLA，我们进一步将头 *in addition to* 分裂，将潜在维度减半；例如，对于 Kimi-K2-TPLA 上的 TP=4，我们在每个设备上使用 32 个磁头  $\times 2$ ，每个磁头具有 320 维的潜在空间。在此设置中，每台设备的计算量减半，而内存流量与 TP=2 匹配；解码仍然受内存限制，因此加速类似于 TP=2。因此，我们报告了两个 GPU 上的测量结果。

Figure 3 configures the maximum batch size at each context length. At a decoding length of 4096, **TPLA** with  $2d_h$  achieves up to  $\sim 2\times$  the throughput of the single-head-latent **MLA** with  $4d_h$ , due to the smaller per-device KV cache. Our parallelization-friendly design raises peak throughput and is resilient under adverse serving loads. At a 32k context length, **DeepSeek-TPLA** is  $1.79\times$  faster than MLA, and **Kimi-K2-TPLA** is  $1.93\times$  faster.

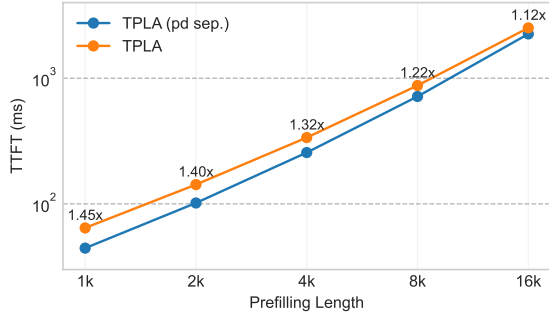


(a) DeepSeek-V3-0324

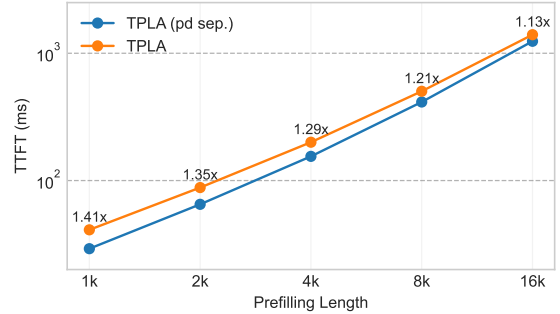


(b) Kimi-K2-Base

Figure 3: Throughput (Decoding) comparing MLA and TPLA.



(a) DeepSeek-V3-0324



(b) Kimi-K2-Base

Figure 4: Latency (TTFT) comparing TPLA and TPLA (pd sep.).

## 5.4.2 Prefilling Latency

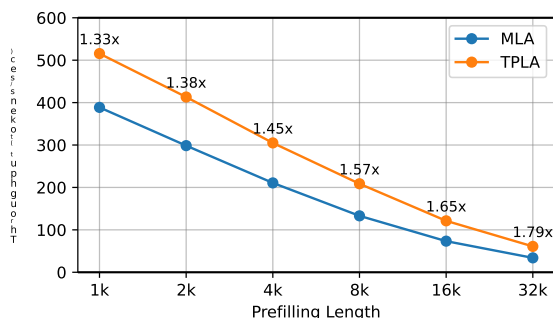
The *prefilling* stage of LLM inference is *compute-bound*. Under TPLA’s TP separation, each device retains the original number of heads, whereas MLA can reduce heads per device by splitting them across devices. As a result, the original TPLA is not ideal for the compute-bound prefill stage. To address this, we introduce **TPLA (sep.)**: it applies the same reparameterization to MLA but *does not* slice RMSNorm or softmax, thereby introducing no approximation error. During prefill, the structure matches MLA: under TP we do not change the latent dimension but partition heads across devices. This significantly reduces per-device compute and alleviates the compute bottleneck.

Figure 4 reports **TTFT** (Time to First Token) on two GPUs for MoE-removed DeepSeek-V3-0324 and Kimi-K2-Base. At a 1K prompt length, **TPLA (sep.)** is  $1.4\times$  faster than TPLA for both models. Given its accuracy-friendly design, this  $1.4\times$  gain is essentially a “free lunch.”

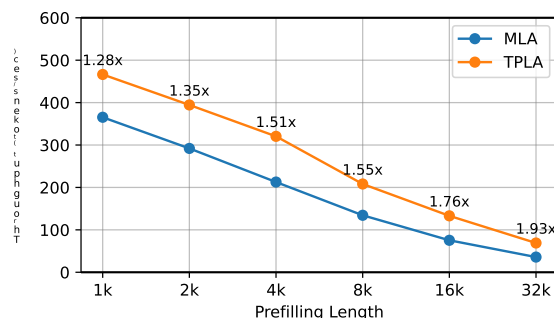
## 6 Conclusion, Limitation and Future Work

We introduce TPLA, which combines the KV cache compression efficiency of MLA with strong compatibility for Tensor Parallelism. It can directly inherit checkpoints from MLA-pretrained models. With two proposed reparameterization techniques, it substantially reduces the loss incurred by converting the attention formulation; combined with PD separation, the training-free conversion error can be driven to a very small level. We evaluate TPLA on commonsense reasoning tasks and the more challenging LongBench benchmark, finding that it preserves the original model’s performance well. Extensive ablations confirm the effectiveness of our TP slicing and reparameterization designs.

图 3 配置了每个上下文长度的最大批量大小。在解码长度为 4096 时，由于每设备 KV 缓存较小，具有  $2d_h$  的 TPLA 实现的吞吐量高达具有  $4d_h$  的单头潜在 MLA 的  $\sim 2\times$ 。我们的并行化友好设计提高了峰值吞吐量，并且在不利服务负载下具有弹性。在 32k 上下文长度下，DeepSeek-TPLA 比 MLA 快  $1.79\times$ ，Kimi-K2-TPLA 比 MLA 快  $1.93\times$ 。

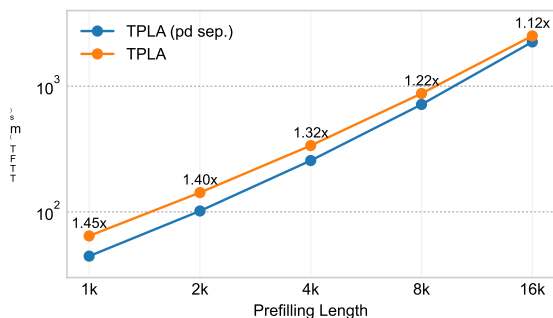


(a) DeepSeek-V3-0324

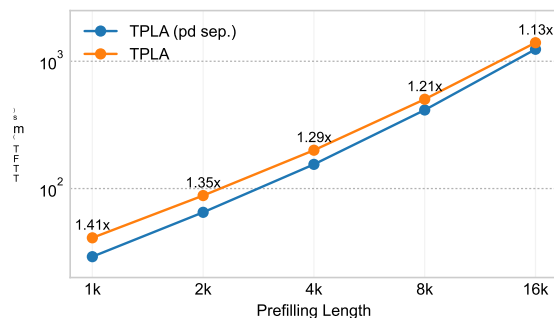


(b) Kimi-K2-Base

图 3: 整个过程（解码）比较 MLA 和 TPLA



(a) DeepSeek-V3-0324



(b) Kimi-K2-Base

图 4: 延迟 (TTFT) 比较 TPLA 和 TPLA (pd sep.)。

#### 5.4.2 预填充延迟

LLM 推理的 *prefilling* 阶段是 *compute-bound*。在 TPLA 的 TP 分离下，每个设备保留原始磁头数，而 MLA 可以通过在设备之间拆分来减少每个设备的磁头数。因此，原始 TPLA 对于计算密集型预填充阶段并不理想。为了解决这个问题，我们引入了 TPLA (九月)：它对 MLA 应用相同的重新参数化，但使用 *does not* 切片 RMSNorm 或 softmax，从而不会引入近似误差。在预填充期间，结构与 MLA 匹配：在 TP 下，我们不更改潜在维度，而是跨设备分区磁头。这显著减少了每台设备的计算量并缓解了计算瓶颈。

图 4 报告了 MoE 移除的 DeepSeek-V3-0324 和 Kimi-K2-Base 的两个 GPU 上的 TTFT（首次令牌时间）。在 1K 提示长度下，两种模型的 TPLA (sep.) 都比 TPLA 快  $1.4\times$ 。鉴于其精度友好的设计，这种  $1.4\times$  增益本质上是“免费午餐”。

## 6 结论、局限性和未来工作

我们引入了 TPLA，它结合了 MLA 的 KV 缓存压缩效率和对 Tensor Parallelism 的强大兼容性。它可以直接从 MLA 预训练模型继承检查点。通过提出的两种重新参数化技术，它大大减少了转换注意力公式所带来的损失；与 PD 分离相结合，无需训练的转换误差可以被驱动到非常小的水平。我们在常识推理任务和更具挑战性的 LongBench 基准上评估 TPLA，发现它很好地保留了原始模型的性能。广泛的消融证实了我们的 TP 切片和重新参数化设计的有效性。

TPLA achieves up to  $2\times$  improvement in throughput, and PD separation delivers up to 29% latency reduction. Overall, TPLA shows strong potential as a powerful and efficient replacement for MLA.

**Limitation and Future Work.** Although PCA demonstrates better performance over Hadamard transform, it has inherent limitations. Specifically, PCA concentrates most of the data’s informative content in the first few dimensions, which provide a representative summary of the global structure. In contrast, the later dimensions primarily capture negligible noise and minor variations that contribute minimally to the overall representation. Consequently, TPLA with group-partitions  $g = 2$  can achieve good performance, but when  $g > 2$ , it probably fails to maintain effectiveness. By contrast, numerical-value balancing via orthogonal transforms, particularly the Hadamard transform, tends to be more effective when partitioning into multiple groups. Empirically, inserting a Hadamard transform into the RMSNorm slicing part yields almost no performance degradation. In future work, we will design and evaluate optimized Hadamard-like orthogonal matrices to balance softmax slicing, thereby improving both robustness and scalability. One advantage of TPLA is that it can directly inherit MLA checkpoints, but this also introduces some conversion errors. Our experiments fully validate TPLA’s expressive capacity and speed advantages. In future work, we will post-pretrain DeepSeek-V3, or train a TPLA-based model from scratch, to further demonstrate TPLA’s excellent expressiveness.

## References

- [1] OpenAI. Hello GPT-4o, 2024.
- [2] Anthropic. Claude 3.5 sonnet, 2024.
- [3] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- [4] Alec Radford. Improving language understanding by generative pre-training. 2018.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Chang Chi-Chih, Lin Wei-Cheng, Lin Chien-Yu, Chen Chong-Yan, Hu Yu-Fang, Wang Pei-Shuo, Huang Ning-Chi, Ceze Luis, Abdelfattah Mohamed, S., and Wu and, Kai-Chiang. Palu: Compressing kv-cache with low-rank projection. *arXiv preprint arXiv:2407.21118*, 2024.
- [7] Chang Chi-Chih, Lin Chien-Yu, Akhauri Yash, Lin Wei-Cheng, Wu Kai-Chiang, Ceze Luis, and Abdelfattah Mohamed, S. xkv: Cross-layer svd for kv-cache compression. *arXiv preprint arXiv:2503.18893*, 2025.
- [8] Oren Matanel, Hassid Michael, Yarden Nir, Adi Yossi, and Schwartz Roy. Transformers are multi-state rnns. *arXiv preprint arXiv:2401.06104*, 2024.
- [9] Liu Zirui, Yuan Jiayi, Jin Hongye, Zhong Shaochen, Xu Zhaozhuo, Braverman Vladimir, Chen Beidi, and Hu Xia. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- [10] Zhang Muru, Mishra Mayank, Zhou Zhongzhu, Brandon William, Wang Jue, Kim Yoon, Ragan-Kelley Jonathan, Song Shuaiwen, Leon, Athiwaratkun Ben, and Dao Tri. Ladder-residual: parallelism-aware architecture for accelerating large model inference with communication overlapping. *arXiv preprint arXiv:2501.06589*, 2025.
- [11] Kim Han-Byul, Hoang Duc, Kundu Arnav, Samragh Mohammad, and Cho Minsik. Spd: Sync-point drop for efficient tensor parallelism of large language models. *arXiv preprint arXiv:2502.20727*, 2025.
- [12] Lamprecht Itay, Karnieli Asaf, Hanani Yair, Giladi Niv, and Soudry Daniel. Tensor-parallelism with partially synchronized activations. *arXiv preprint arXiv:2506.19645v1*, 2025.
- [13] Li Qingyuan, Zhang Bo, Ye Liang, Zhang Yifan, Wu Wei, Sun Yerui, Ma Lin, and Xie Yuchen. Flash communication: Reducing tensor parallelization bottleneck for fast large language model inference. *arXiv preprint arXiv:2412.04964*, 2024.
- [14] Smith Shaden, Patwary Mostofa, Norick Brandon, LeGresley Patrick, Rajbhandari Samyam, Casper Jared, Liu Zhun, Prabhumoye Shrimai, Zerveas George, Korthikanti Vijay, Zhang Elton, Child Rewon, Aminabadi Reza, Yazdani, Bernauer Julie, Song Xia, Shoenybi Mohammad, He Yuxiong, Houston Michael, Tiwary Saurabh, and Catanzaro and, Bryan. Using deepspeed and megatron to train megatron-turing nlG 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*, 2022.
- [15] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.

TPLA 的吞吐量提高高达  $2\times$ ，PD 分离可将延迟降低高达 29%。总体而言，TPLA 作为 MLA 的强大而高效的替代品显示出强大的潜力。

限制和未来的工作。尽管 PCA 表现出比 Hadamard 变换更好的性能，但它具有固有的局限性。具体来说，PCA 将大部分数据的信息内容集中在前几个维度，提供了全局结构的代表性摘要。相比之下，后面的维度主要捕获可忽略不计的噪声和对整体表示影响最小的微小变化。因此，具有组分区  $g = 2$  的 TPLA 可以获得良好的性能，但当  $g > 2$  时，它可能无法保持有效性。相比之下，通过正交变换（特别是哈达玛变换）进行数值平衡在划分为多个组时往往更有效。根据经验，将 Hadamard 变换插入 RMSNorm 切片部分几乎不会产生性能下降。在未来的工作中，我们将设计和评估优化的类 Hadamard 正交矩阵来平衡 softmax 切片，从而提高鲁棒性和可扩展性。TPLA 的优点之一是它可以直接继承 MLA 检查点，但这也引入了一些转换错误。我们的实验充分验证了 TPLA 的表达能力和速度优势。在未来的工作中，我们将对 DeepSeek-V3 进行后预训练，或者从头开始训练基于 TPLA 的模型，以进一步展示 TPLA 出色的表达能力。

## 参考

[1] 开放人工智能。你好，GPT-4o，2024。[2] 人类。Claude 3.5 十四行诗，2024。[3] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, 等。Gemini 1.5: 解锁跨数百万个上下文标记的多模式理解。 *arXiv preprint arXiv:2403.05530*, 2024 年。[4] 亚历克·雷德福。通过生成预训练提高语言理解。2018。[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell 等。语言模型是小样本学习者。 *Advances in neural information processing systems*, 33:1877–1901, 2020。[6] 张志智、林伟成、林建宇、陈重彦、胡玉芳、王培硕、黄宁吉、Ceze Luis, Abdelfattah Mohamed, S.、吴和凯强。Palu: 使用低秩投影压缩 kv-cache。 *arXiv preprint arXiv:2407.21118*, 2024。[7] Chang Chi-Chih, Lin Chien-Yu, Akhauri Yash, Lin Wei-Cheng, Wu Kai-Chiang, Ceze Luis 和 Abdelfattah Mohamed, S. xkv: 用于 kv 缓存压缩的跨层 svd。 *arXiv preprint arXiv:2503.18893*, 2025。[8] Oren Matanel, Hassid Michael, Yarden Nir, Adi Yossi 和 Schwartz Roy。Transformer 是多状态 rnn。 *arXiv preprint arXiv:2401.06104*, 2024。[9] 刘子瑞, 袁嘉义, 金红叶, 钟绍辰, 徐兆卓, Braverman Vladimir, 陈蓓迪, 胡夏。Kivi: kv 缓存的免调整非对称 2 位量化。 *arXiv preprint arXiv:2402.02750*, 2024。[10] 张木如, Mishra Mayank, 周中柱, Brandon William, 王珏, Kim Yoon, Ragan-Kelley Jonathan, 宋帅文, Leon, Athiwaratkun Ben, Dao Tri。阶梯残差: 并行感知架构, 用于通过通信重叠加速大型模型推理。 *arXiv preprint arXiv:2501.06589*, 2025。[11] Kim Han-Byul, Hoang Duc, Kundu Arnav, Samragh Mohammad 和 Cho Minsik。Spd: 同步点下降, 以实现大型语言模型的高效张量并行性。 *arXiv preprint arXiv:2502.20727*, 2025。[12] Lamprecht Itay, Karnieli Asaf, Hanani Yair, Giladi Niv 和 Soudry Daniel。具有部分同步激活的张量并行性。 *arXiv preprint arXiv:2506.19645v1*, 2025。[13] 李清源, 张波, 叶亮, 张一凡, 吴伟, 孙业瑞, 马琳, 谢雨辰。Flash 通信: 减少快速大型语言模型推理的张量并行化瓶颈。 *arXiv preprint arXiv:2412.04964*, 2024。[14] Smith Shaden, Patwary Mostofa, Norick Brandon, Le Gresley Patrick, Rajbhandari Samyam, Casper Jared, Liu Zhun, Prabhumoye Shrimai, Zerveas George, Korthikanti Vijay, Zhang Elton, Child Rewon, Aminabadi Reza, Yazdani, Bernauer Julie, 宋霞, Shoeybi Mohammad, 何宇雄, Houston Michael, Tiwary Saurabh, Catanzaro 和 Bryan。使用 deepspeed 和 megatron 训练大型生成语言模型 megatron-turing nlG 530b。 *arXiv preprint arXiv:2201.11990*, 2022 年。

[15] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón 和 Sumit Sanghai。Gqa: 从多头检查点训练广义多查询变压器模型。 *arXiv preprint arXiv:2305.13245*, 2023 年。

- [16] Fanxu Meng, Pingzhi Tang, Zengwei Yao, and Muhan Zhang. Transmla: Multi-head latent attention is all you need. *arXiv preprint arXiv:2502.07864*, 2025.
- [17] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *CoRR*, abs/2405.04434, 2024.
- [18] AI@Meta. Llama 3 model card, 2024.
- [19] Aixiu Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [20] Zadouri Ted, Strauss Hubert, and Dao Tri. Hardware-efficient attention for fast decoding. *arXiv preprint arXiv:2505.21487v1*, 2025.
- [21] Ge Suyu, Zhang Yunan, Liu Liyuan, Zhang Minjia, Han Jiawei, and Gao Jianfeng. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.
- [22] Zhou Xiabin, Wang Wenbin, Zeng Minyan, Guo Jiaxian, Liu Xuebo, Shen Li, Zhang Min, and Ding Liang. Dynamickv: Task-aware adaptive kv cache compression for long context llms. *arXiv preprint arXiv:2412.14838*, 2024.
- [23] Lin Xiaolin, Wang Jingcun, Kondrateva Olga, Shi Yiyu, Li Bing, and Zhang Grace, Li. Compresskv: Semantic retrieval heads know what tokens are not important before generation. *arXiv preprint arXiv:2508.02401v1*, 2025.
- [24] Li Yuhong, Huang Yingbing, Yang Bowen, Venkitesh Bharat, Locatelli Acyr, Ye Hanchen, Cai Tianle, Lewis Patrick, and Chen Deming. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- [25] Fu Qichen, Cho Minsik, Merth Thomas, Mehta Sachin, Rastegari Mohammad, and Najibi Mahyar. Lazyllm: Dynamic token pruning for efficient long context llm inference. *arXiv preprint arXiv:2407.14057*, 2024.
- [26] Zhang Zhenyu, Sheng Ying, Zhou Tianyi, Chen Tianlong, Zheng Lianmin, Cai Ruisi, Song Zhao, Tian Yuandong, Ré Christopher, Barrett Clark, Wang Zhangyang, and Chen Beidi. H<sub>2</sub>O: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023.
- [27] Wang Zheng, Jin Boxiao, Yu Zhongzhi, and Zhang Minjia. Model tells you where to merge: Adaptive kv cache merging for llms on long-context tasks. *arXiv preprint arXiv:2407.08454*, 2024.
- [28] Liu Xin, Liu Pei, and Tang Guoming. Zsmerge: Zero-shot kv cache compression for memory-efficient long-context llms. *arXiv preprint arXiv:2503.10714*, 2025.
- [29] Hu Jie, Wang Shengnan, He Yutong, Gong Ping, Yi Jiawei, Zhang Juncheng, Bai Youhui, Chen Renhai, Zhang Gong, Li Cheng, and Yuan Kun. Efficient long-context llm inference via kv cache clustering. *arXiv preprint arXiv:2506.11418*, 2025.
- [30] Yang Yifei, Cao Zouying, Chen Qiguang, Qin Libo, Yang Dongjie, Zhao Hai, and Chen Zhi. Kvsharer: Efficient inference via layer-wise dissimilar kv cache sharing. *arXiv preprint arXiv:2410.18517*, 2024.
- [31] Rajput Shashank, Sheng Ying, Owen Sean, and Chiley Vitaliy. Inference-friendly models with mixattention. *arXiv preprint arXiv:2409.15012*, 2024.
- [32] Wu Haoyi and Tu Kewei. Layer-condensed kv cache for efficient inference of large language models. *arXiv preprint arXiv:2405.10637*, 2024.
- [33] Wu You, Wu Haoyi, and Tu Kewei. A systematic study of cross-layer kv sharing for efficient llm inference. *arXiv preprint arXiv:2410.14442*, 2024.
- [34] Brandon William, Mishra Mayank, Nrusimha Aniruddha, Panda Rameswar, and Kelly Jonathan, Ragan. Reducing transformer key-value cache size with cross-layer attention. *arXiv preprint arXiv:2405.12981*, 2024.
- [35] Zhang Rongzhi, Wang Kuang, Liu Liyuan, Wang Shuohang, Cheng Hao, Zhang Chao, and Shen Yelong. Lorc: Low-rank compression for llms kv cache with a progressive compression strategy. *arXiv preprint arXiv:2410.03111*, 2024.
- [36] Lin Bokai, Zeng Zihao, Xiao Zipeng, Kou Siqi, Hou Tianqi, Gao Xiaofeng, Zhang Hao, and Deng Zhijie. Matryoshkakv: Adaptive kv compression via trainable orthogonal projection. *arXiv preprint arXiv:2410.14731*, 2024.
- [37] Yu Hao, Yang Zelan, Li Shen, Li Yong, and Wu Jianxin. Effectively compress kv heads for llm. *arXiv preprint arXiv:2406.07056*, 2024.
- [38] Hooper Coleman, Kim Sehoon, Mohammadzadeh Hiva, Mahoney Michael, W., Shao Yakun, Sophia, Keutzer Kurt, and Gholami Amir. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[16] 孟凡旭, 唐平志, 姚增伟, 张慕涵. Transmla: 多头潜在注意力就是你所需要的. *arXiv preprint arXiv:2502.07864*, 2025. [17] DeepSeek-AI. Deepseek-v2: 强大、经济且高效的专家混合语言模型. *CoRR*, abs/2405.04434, 2024. [18] AI@Meta. Llama 3模型卡, 2024年. [19] 刘爱新, 冯北, 薛冰, 王兵轩, 吴博超, 陆成达, 赵成刚, 邓成奇, 张晨宇, 阮冲, 等. Deepseek-v3 技术报告. *arXiv preprint arXiv:2412.19437*, 2024 年. [20] Zadouri Ted, Strauss Hubert 和 Dao Tri. 硬件高效关注快速解码. *arXiv preprint arXiv:2505.21487v1*, 2025. [21] 葛素玉, 张玉楠, 刘丽媛, 张敏佳, 韩家伟, 高剑峰. 模型告诉您要丢弃什么: llms 的自适应 kv 缓存压缩. *arXiv preprint arXiv:2310.01801*, 2023. [22] 周夏斌, 王文斌, 曾敏言, 郭家贤, 刘学波, 沉力, 张敏, 丁亮. Dynamickv: 用于长上下文 llms 的任务感知自适应 kv 缓存压缩. *arXiv preprint arXiv:2412.14838*, 2024. [23] 林晓琳, 王景存, Kondrateva Olga, 石一字, 李兵, 张格蕾丝, 李. compresskv: 语义检索头在生成之前就知道哪些标记不重要. *arXiv preprint arXiv:2508.02401v1*, 2025. [24] 李玉红, 黄英兵, 杨博文, Venkitesh Bharat, Locatelli Acyr, 叶汉臣, 蔡天乐, Lewis Patrick, 陈德明. Snapkv: LLM 在一代之前就知道你在寻找什么. *arXiv preprint arXiv:2404.14469*, 2024. [25] Fu Qichen, Cho Minsik, Merth Thomas, Mehta Sachin, Rastegari Mohammad 和 Najibi Mahyar. Lazyllm: 动态标记修剪, 用于高效的长上下文 llm 推理. *arXiv preprint arXiv:2407.14057*, 2024. [26] 张振宇, 盛英, 周天一, 陈天龙, 郑连民, 蔡瑞思, 宋钊, 田元东, Ré Christopher, Barrett Clark, 王张阳, 陈蓓迪. H<sub>2</sub>O: 用于大型语言模型高效生成推理的重量级预言机. *arXiv preprint arXiv:2306.14048*, 2023. [27] 王征, 金伯孝, 余忠志, 张敏佳. 模型告诉您在哪儿合并: 针对长上下文任务的 llms 的自适应 kv 缓存合并. *arXiv preprint arXiv:2407.08454*, 2024. [28] 刘鑫, 刘培, 唐国明. Zsmmerge: 零样本 kv 缓存压缩, 用于内存高效的长上下文 llms. *arXiv preprint arXiv:2503.10714*, 2025. [29] 胡杰, 王胜男, 何雨桐, 龚平, 易嘉伟, 张俊成, 白友辉, 陈仁海, 张工, 李成, 袁坤. 通过 kv 缓存集群进行高效的长上下文 llm 推理. *arXiv preprint arXiv:2506.11418*, 2025. [30] 杨一飞, 曹奏英, 陈其光, 秦立波, 杨东杰, 赵海, 陈志. Kvsharer: 通过分层不同的 kv 缓存共享进行高效推理. *arXiv preprint arXiv:2410.18517*, 2024 年. [31] Rajput Shashank, Sheng Ying, Owen Sean 和 Chiley Vitaliy. 具有混合注意力的推理友好模型. *arXiv preprint arXiv:2409.15012*, 2024. [32] 吴浩一, 屠可伟. 层压缩的 kv 缓存, 用于大型语言模型的高效推理. *arXiv preprint arXiv:2405.10637*, 2024. [33] 吴悠, 吴浩一, 屠可伟. 用于高效 llm 推理的跨层 kv 共享的系统研究. *arXiv preprint arXiv:2410.14442*, 2024 年. [34] Brandon William, Mishra Mayank, Nrusimha Aniruddha, Panda Rameswar 和 Kelly Jonathan, Ragan. 通过跨层关注减少转换器键值缓存大小. *arXiv preprint arXiv:2405.12981*, 2024. [35] 张荣植, 王匡, 刘立源, 王硕航, 程浩, 张超, 沉业龙. Lorc: 采用渐进压缩策略的 llms kv 缓存的低阶压缩. *arXiv preprint arXiv:2410.03111*, 2024. [36] 林博凯, 曾子豪, 肖子鹏, 寇思琪, 侯天琪, 高晓峰, 张浩, 邓志杰. Matryoshkakov: 通过可训练的正交投影进行自适应 kv 压缩. *arXiv preprint arXiv:2410.14731*, 2024. [37] 余浩, 杨泽兰, 李深, 李勇, 吴建新. 有效压缩 llm 的 kv 头. *arXiv preprint arXiv:2406.07056*, 2024 年. [38] Hooper Coleman, Kim Sehoon, Mohammadzadeh Hiva, Mahoney Michael, W., Shao Yakun, Sophia, Keutzer Kurt 和 Gholami Amir. Kvquant: 通过 kv 缓存量化实现 1000 万上下文长度 llm 推理. *arXiv preprint arXiv:2401.18079*, 2024 年.

- [39] Wang Zongwu, Xu Peng, Liu Fangxin, Hu Yiwei, Sun Qingxiao, Li Gezi, Li Cheng, Wang Xuan, Jiang Li, and Guan Haibing. Million: Mastering long-context llm inference via outlier-immunized kv product quantization. *arXiv preprint arXiv:2504.03661*, 2025.
- [40] Dong Shichen, Cheng Wen, Qin Jiayu, and Wang Wei. Qaq: Quality adaptive quantization for llm kv cache. *arXiv preprint arXiv:2403.04643*, 2024.
- [41] Yao Dingyu, Shen Bowen, Lin Zheng, Liu Wei, Luan Jian, Wang Bin, and Wang Weiping. Tailorkv: A hybrid framework for long-context inference via tailored kv cache optimization. *arXiv preprint arXiv:2505.19586*, 2025.
- [42] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. Large scale distributed deep networks. *Advances in neural information processing systems*, 25, 2012.
- [43] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.
- [44] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyounJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [45] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM symposium on operating systems principles*, pages 1–15, 2019.
- [46] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [47] Xu Qifan, Li Shenggui, Gong Chaoyu, and You Yang. An efficient 2d method for training super-large deep learning models. *arXiv preprint arXiv:2104.05343*, 2021.
- [48] Bian Zhengda, Xu Qifan, Wang Boxiang, and You Yang. Maximizing parallelism in distributed training for huge neural networks. *arXiv preprint arXiv:2105.14450*, 2021.
- [49] Li Shenggui, Xue Fuzhao, Baranwal Chaitanya, Li Yongbin, and You Yang. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120*, 2021.
- [50] Amr Elmeleegy, Harry Kim, David Zier, Kyle Kranen, Neelay Shah, Ryan Olson, and Omri Kahalon. NVIDIA dynamo, a low-latency distributed inference framework for scaling reasoning ai models. NVIDIA Developer Blog, March 2025. Published March 18, 2025.
- [51] Juntao Zhao, Jiuru Li, and Chuan Wu. Sandwich: Separating prefill-decode compilation for efficient cpu llm serving. *arXiv preprint arXiv:2507.18454*, 2025.
- [52] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.
- [53] DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. *CoRR*, abs/2401.02954, 2024.
- [54] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [55] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.
- [56] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. *CoRR*, abs/1803.05457, 2018.
- [57] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: reasoning about physical commonsense in natural language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 7432–7439. AAAI Press, 2020.
- [58] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In Anna Korhonen, David R. Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 4791–4800. Association for Computational Linguistics, 2019.

[39] 王宗武, 徐鹏, 刘方新, 胡一伟, 孙庆晓, 李格子, 李成, 王选, 蒋莉, 关海兵。Million: 通过异常值免疫 kv 产品量化掌握长上下文 llm 推理。 *arXiv preprint arXiv:2504.03661*, 2025. [40] 董世臣, 程文, 秦嘉玉, 王伟。Qaq: llm kv 缓存的质量自适应量化。 *arXiv preprint arXiv:2403.04643*, 2024. [41] 姚定宇, 沉博文, 林正, 刘伟, 栾健, 王斌, 王卫平。Tailorkv: 通过定制的 kv 缓存优化进行长上下文推理的混合框架。 *arXiv preprint arXiv:2505.19586*, 2025. [42] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc' aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang 等。大规模分布式深度网络。 *Advances in neural information processing systems*, 25, 2012. [43] 亚历山大·谢尔盖耶夫和迈克·德尔·巴尔也。Horovod: 张量流中快速、简单的分布式深度学习。 *arXiv preprint arXiv:1802.05799*, 2018. [44] 黄艳平, 程友龙, Ankur Bapna, Orhan Firat, 陈德豪, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, 等。Gpipe: 使用管道并行性对巨型神经网络进行高效训练。 *Advances in neural information processing systems*, 32, 2019. [45] Deepak Narayanan, Aaron Harlap, Amar Phanishaye, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons 和 Matei Zaharia。Pipedream: 用于 DNN 训练的广义管道并行性。见 *Proceedings of the 27th ACM symposium on operating systems principles*, 2019 年第 1-15 页。[46] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper 和 Bryan Catanzaro。Megatron-lm: 使用模型并行性训练数十亿参数语言模型。 *arXiv preprint arXiv:1909.08053*, 2019. [47] 徐起帆, 李胜贵, 龚超宇, 尤阳。一种训练超大型深度学习模型的高效二维方法。 *arXiv preprint arXiv:2104.05343*, 2021. [48] 卞正大, 徐奇帆, 王伯祥, 尤阳。最大化大型神经网络分布式训练的并行性。 *arXiv preprint arXiv:2105.14450*, 2021. [49] 李胜贵, 薛福照, Baranwal Chaitanya, 李永斌, 尤阳。序列并行: 系统角度的长序列训练。 *arXiv preprint arXiv:2105.13120*, 2021 年。[50] Amr Elmeleegy, Harry Kim, David Zier, Kyle Kranen, Neelam Shah, Ryan Olson 和 Omri Kahalon。NVIDIA dynamo, 一种用于扩展推理人工智能模型的低延迟分布式推理框架。NVIDIA 开发者博客, 2025 年 3 月。2025 年 3 月 18 日发布。[51] Juntao Zhao, Jiuru Li 和 Chuan Wu。三明治: 分离预填充-解码编译以实现高效的 cpu llm 服务。 *arXiv preprint arXiv:2507.18454*, 2025. [52] 钟银民, 刘胜宇, 陈俊达, 胡建波, 朱一波, 刘轩哲, 金鑫, 张浩。{DistServe}: 分解预填充和解码, 以实现良好输出优化的大型语言模型服务。 *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 第 193-210 页, 2024 年。[53] DeepSeek-AI。Deepseek LLM: 以长期主义扩展开源语言模型。 *CoRR*, abs/2401.02954, 2024. [54] Kimi 团队, 白一凡, 包一平, 陈冠铎, 陈家豪, 陈宁馨, 陈瑞爵, 陈彦茹, 陈元坤, 陈玉田, 等。Kimi k2: 开放代理智能。 *arXiv preprint arXiv:2507.20534*, 2025. [55] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song 和 Jacob Steinhardt。测量大规模多任务语言理解。在 *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021* 中。OpenReview.net, 2021 年。[56] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick 和 Oyvind Tafjord。您认为您已经解决了问答问题吗? 尝试 arc, AI2 推理挑战赛。 *CoRR*, abs/1803.05457, 2018. [57] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Taka 和 Yejin Choi。PIQA: 用自然语言推理物理常识。在 *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, 第 7432-7439 页。AAAI 出版社, 2020 年。[58] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi 和 Yejin Choi。Hellaswag: 机器真的能完成你的句子吗? 载于 Anna Korhonen, David R. Traum 和 Lluís Màrquez, 编辑, *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, 第 4791-4800 页。计算语言学协会, 2019。

- [59] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? A new dataset for open book question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 2381–2391. Association for Computational Linguistics, 2018.
- [60] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, 2021.
- [61] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- [62] Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. Smolm-corporus. 2024.
- [63] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

[59]托多尔·米哈伊洛夫、彼得·克拉克、图沙尔·科特和阿什什·萨巴瓦尔。铠甲可以导电吗？用于开卷问答的新数据集。载于 Ellen Riloff、David Jiang、Julia Hockenmaier 和 Jun’ ichi Tsujii, 编辑, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, 第 2381-2391 页。计算语言学协会, 2018。[60] Keisuke Sakaguchi、Ronan Le Bras、Chandra Bhagavatula 和 Yejin Choi。Winogrande: 大规模的对抗性 winograd 模式挑战。 *Commun. ACM*, 64(9):99–106, 2021。[61] Stephen Merity、Caiming Xiong、James Bradbury 和 Richard Socher。指针哨兵混合模型, 2016。[62] Loubna Ben Allal、Anton Lozhkov、Guilherme Penedo、Thomas Wolf 和 Leandro von Werra。斯莫尔姆语料库。2024。[63] 白雨诗, 吕鑫, 张家杰, 吕宏昌, 唐建凯, 黄志典, 杜正晓, 刘晓, 曾敖汉, 侯磊, 等。Longbench: 用于长上下文理解的双语、多任务基准。 *arXiv preprint arXiv:2308.14508*, 2023 年。